



UNIVERSITY OF SALERNO

Department of Computer Science

Master Degree in Computer Science

THESIS

Prompt Syntax Tree: A Formal Representation for the Analysis and Automatic Decompositions of LLM Prompts

SUPERVISOR

Prof. Palomba Fabio

Dott. Cannavale Alfonso

University of Salerno

CANDIDATE

Giovanni Scorziello

Matr.: 0522501701

Academic Year 2024-2025

This thesis was carried out in

sesə^{lab}
SOFTWARE ENGINEERING
SALERNO

To those who look down on me and to those who have been by my side

Abstract

The widespread adoption of Large Language Models (LLMs) has established Prompt Engineering as a critical capability for obtaining reliable and accurate outputs. However, prompts are frequently designed in an ad-hoc manner, treated as unstructured strings without a clear underlying architecture, which makes them difficult to analyze, version, and systematically improve. This thesis addresses this engineering gap by proposing the **Prompt Syntax Tree** (PST), a formal hierarchical representation that decomposes prompts into their constituent semantic components. Specifically, the work presents a workflow that transitions from the analysis of advanced prompting paradigms, including Zero-Shot, Few-Shot, Chain-of-Thought (CoT), and Tree-of-Thoughts (ToT), to the development of an automated extraction tool.

These paradigms are formalized into a structured schema comprising sections such as context, instructions, constraints, and reasoning paths. To operationalize this framework, an annotated dataset was constructed to train a fine-tuned language model capable of automatically segmenting raw text into valid PST structures. The empirical evaluation demonstrates the model's effectiveness in producing structured JSON representations with structural validity and align to the original user intent. This work offers a practical and conceptual contribution to the field of "Promptware", providing the necessary tooling to support the rigorous analysis and optimization of LLM interactions.

Contents

List of Figures	v
List of Tables	vii
1 Introduction	1
1.1 Context	1
1.2 Motivations and Objectives	2
1.3 Results	3
1.4 Document Structure	4
2 State of the Art	5
2.1 Prompt Engineering	5
2.1.1 Historical Evolution	6
2.1.2 The Shift to Semantic Interaction	6
2.1.3 Impact on Software Development	7
2.1.4 The Increasing Value of the Prompt	8
2.2 Prompt Engineering Fundamentals	8
2.2.1 Overview of Current Paradigms	10
2.2.2 The Gap Between Theory and Practice	10
2.3 The Engineering Gap in Prompt Development	11
2.3.1 The Fragility of Unstructured Prompts	11

2.3.2	The Limitations of “Trial and Error”	12
2.3.3	The Need for Systematic Engineering	12
2.4	The Unstructured Data Problem	13
2.4.1	The Absence of Structural Representation	13
2.4.2	The Reproducibility Crisis	14
2.4.3	Lack of Granular Control	15
3	The Prompt Syntax Tree (PST)	17
3.1	Introduction	17
3.2	Prompt Taxonomy	18
3.2.1	Zero-Shot Prompting	18
3.2.2	Few-Shot Prompting	19
3.2.3	Role Playing Prompting	20
3.2.4	Chain-of-Thought (CoT) Prompting	20
3.2.5	Zero-Shot Chain-of-Thought Prompting	22
3.2.6	Tree-of-Thoughts (ToT) Prompting	24
3.2.7	Graph-of-Thought (GoT) Prompting	25
3.2.8	Self-Refine Prompting	27
3.2.9	Emotion Prompting	28
3.3	Formalizing the Prompt Structure	30
3.3.1	From Linear String to Hierarchical Representation	30
3.3.2	The AST Analogy	31
3.4	Derivation of the PST Components	31
3.4.1	Core Components	31
3.4.2	Shot-Based Components	33
3.4.3	Reasoning-Based Components	33
3.4.4	Robustness to Advanced Prompting Techniques	34
3.5	Formal Specification	35
3.5.1	Hierarchical Structure	35
3.5.2	Leaf Nodes and Sequence Preservation	38
3.5.3	Construction Strategy: Inference and Alignment	39
3.6	Data Representation	39

3.6.1	JSON Schema Definition	40
3.7	Summary	41
4	Implementation and Model Training	42
4.1	Introduction	42
4.2	Overview of the Training Pipeline	43
4.3	Dataset Acquisition and Preparation	43
4.3.1	Source Selection	43
4.3.2	Single-Turn Extraction Strategy	43
4.3.3	Data Cleaning	44
4.4	Data Annotation and Augmentation	44
4.4.1	Labeling Process and Annotation Guidelines	44
4.4.2	Synthetic Data Generation	46
4.4.3	JSON Serialization	48
4.4.4	Dataset Evaluation and Quality Assurance	48
4.5	Model Selection	49
4.5.1	Semantic Disambiguation Capabilities	49
4.5.2	Complex Instruction Following	50
4.5.3	Computational Feasibility	50
4.6	Training Methodology	50
4.6.1	Chat Format and System Prompt	50
4.6.2	Hyperparameters and Configuration	53
4.7	Preliminary Results	54
5	Empirical Evaluation	55
5.1	Objectives	55
5.2	Context of the Study	56
5.3	Data Collection Procedure	57
5.4	Evaluation Metrics	58
5.4.1	Analysis for RQ_1	58
5.4.2	5.4.2 Analysis for RQ_2	59
5.4.3	Analysis for RQ_3	60

6	Analysis of the Results	62
6.1	Results for RQ_1	62
6.1.1	Structural Reliability and JSON Validity	63
6.1.2	Content Reliability and Coverage	63
6.2	Results for RQ_2	64
6.3	Results for RQ_3	65
6.3.1	The “Format vs. Constraint” Challenge	65
6.3.2	Confusion Matrix and Qualitative Analysis	65
7	Threats to Validity	69
7.1	Construct Validity	69
7.1.1	Expressiveness of the PST Taxonomy	69
7.1.2	Subjectivity in Ground Truth Generation.	70
7.2	Internal Validity	70
7.2.1	Model Scale and Semantic Depth	70
7.2.2	Synthetic Data Artifacts	71
7.3	External Validity	71
7.3.1	Morphological and Typological Incompatibility	71
7.3.2	Empirical Validation Scope (Software Engineering)	72
7.4	Summary	72
8	Conclusions	73
8.1	Summary	73
8.2	Implications and Contributions	74
8.3	Limitations and Future Work	75
	Bibliography	77

List of Figures

3.1	Example of zero-shot prompt from the site promptingguide.ai	18
3.2	Example of few-shot prompt from Brown et al. [1]	19
3.3	Example of one-shot prompt from Brown et al. [1]	20
3.4	Example of role playing prompt from Kong et al. [2]	21
3.5	Example of Chain-of-Thought prompt from Wei et al. [3]	22
3.6	trigger phrase selection from Kojima et al. [4]	23
3.7	Example of Zero-Shot Chain-of-Thought prompt from Kojima et al. [4]	23
3.8	Tree-of-Thoughts representation from Yao et al. [5]	24
3.9	Tree-of-Thoughts representation from the site promptingguide.ai . .	25
3.10	Graph of Thoughts deduction example from Besta et al. [6]	25
3.11	Self-Refine prompting example from Madaan et al. [7]	28
3.12	Emotion Prompting example from Li et al. [8]	29
3.13	Example of base structure	33
3.14	Example of a structure with shot-based components	33
3.15	Example of structure with “think” components	34
3.16	Example prompt used to illustrate logical aggregation.	36
3.17	Examples of Structure	36
3.18	Prompt Syntax Tree Structure	37
3.19	Prompt Syntax Tree leaf definition	38

3.20 Prompt Syntax Tree JSON Object	40
---	----

List of Tables

4.1	Examples of Prompts used as Seed for the generation of synthetic data	47
6.1	Structural validity and faithfulness metrics for RQ_1	63
6.2	Segmentation performance (field-level F1) for RQ_2	64
6.3	Semantic disambiguation performance (field-level F1) for RQ_3	65

CHAPTER 1

Introduction

This chapter provides the general context of the dissertation and highlighting the paradigm shift in human-computer interaction brought about by Large Language Models (LLMs) and the emerging discipline of **Prompt Engineering**. The main problem addressed by this thesis, the lack of a formal, structured representation for prompts, is analyzed, and the specific objectives of the work are defined. Finally, the main results obtained are presented, and the overall structure of the document is outlined.

1.1 Context

The widespread adoption of Large Language Models (LLMs), such as ChatGPT, Gemini, and Llama, has fundamentally altered the landscape of problem-solving and software development. As demonstrated by Brown et al. [1], these models possess **In-Context Learning** capabilities, allowing them to perform complex tasks based on natural language instructions without the need for weight updates. This transition marks a move away from rigid, syntax-based command execution toward a model of semantic interaction, where the user's intent acts as the primary driver of computational output.

In this landscape, Prompt Engineering has emerged as a critical discipline. It is no longer sufficient to simply ask a question; users must act as *semantic architects*, designing prompts that bridge the gap between abstract human intent and the model’s execution. The literature has converged on a standard anatomy for these prompts, identifying key components such as Instructions, Context, Input Data, and Output Indicators. Furthermore, advanced paradigms have been developed to handle complex reasoning, ranging from **Few-Shot** prompting to **Chain-of-Thought** (CoT) and **Tree-of-Thoughts** (ToT).

However, despite the increasing value of the prompt as a “program”, the current engineering approach remains immature. In the software development lifecycle, where LLMs are now integral for code generation and debugging, the reliability of these interactions is paramount. Yet, the interaction often remains a stochastic process of trial and error, lacking the rigor required for professional engineering.

1.2 Motivations and Objectives

Currently, the fundamental limitation hindering the maturation of Prompt Engineering is the absence of a formal **Data Structure** to represent prompts. While concepts like “Promptware” treat prompts as engineering artifacts, their underlying representation remains a flat, unstructured sequence of characters.

This “string-based” paradigm creates a disconnect between the conceptual model of a prompt (which has distinct logical parts like context, constraints, and reasoning paths) and its physical representation. Unlike traditional software, where code is parsed into **Abstract Syntax Trees** (AST) for analysis, prompts lack a standardized format that allows for granular control, automated validation, or versioning of specific components.

This work addresses this gap by proposing a unified solution: the **Prompt Syntax Tree** (PST). Drawing an analogy with compiler theory, this thesis aims to formalize a hierarchical representation that decomposes prompts into addressable semantic units.

The primary objectives of this study are:

- **Formalize the Prompt Structure:** define a comprehensive taxonomy and hierarchical schema (PST) that captures both standard prompt components and advanced reasoning patterns (e.g., CoT, ToT).
- **Construct a PST-Aligned Dataset:** create a supervised corpus of human-annotated prompts, mapping raw unstructured text to structured PST JSON objects.
- **Develop an Automated Parsing Tool:** train and evaluate a Fine-Tuned Language Model (Llama-3-8B) capable of automatically decomposing raw prompts into their PST representation with high structural validity.

1.3 Results

The study produced several significant results, validating both the theoretical framework and the practical implementation of the automated parser. The main findings are as follows:

- **Structural Reliability:** The fine-tuned model demonstrated stability, achieving a 100% JSON Validity Rate across the test set. This confirms that the model successfully learned the strict syntactic rules of the PST schema.
- **Lossless Representation:** The framework achieved a Coverage Score of 99.91% with a negligible Hallucination Rate (0.18%). This indicates that the PST can represent diverse prompting paradigms without losing the user’s original intent or semantic content.
- **Semantic Segmentation:** The tool proved highly effective in isolating core components, particularly distinguishing Examples ($F1 = 1.0$) and Data ($F1 = 0.88$) from the main instructions. However, the analysis also highlighted challenges in disambiguating subtle semantic categories, specifically between formatting instructions and negative constraints.

The developed framework contributes to the field of “Promptware Engineering” by providing the first necessary step toward automated prompt analysis, linting, and optimization tools.

1.4 Document Structure

This thesis is organized into eight chapters, guiding the reader from the theoretical foundations to the empirical validation of the proposed framework.

- **Chapter 1:** Introduction, provides the context of the study, defining the problem of unstructured prompting, the objectives of the PST, and a summary of the key contributions.
- **Chapter 2:** State of the Art, explores the evolution of Prompt Engineering, analyzing current paradigms (Zero-Shot, CoT, etc.) and discussing the “Engineering Gap” and reproducibility crisis that motivate this work.
- **Chapter 3:** The Prompt Syntax Tree (PST) defines the theoretical contribution. It formalizes the hierarchical structure of the PST, detailing the derivation of nodes (Context, Reasoning, etc.) and the JSON serialization schema.
- **Chapter 4:** Implementation and Model Training details the engineering pipeline. It covers the creation of the dataset derived from DevGPT, the synthetic data augmentation strategy, and the fine-tuning process of the Llama-3 model.
- **Chapter 5:** Empirical Evaluation outlines the research methodology. It defines the research questions (RQ_1 to RQ_3) and the specific metrics used for validation, such as Structural Validity and Field-Level F1 scores.
- **Chapter 6:** Analysis of the Results presents the quantitative findings. It answers the research questions by analyzing the model’s performance in generating valid PSTs and its ability to disambiguate semantic components.
- **Chapter 7:** Threats to Validity examines the limitations of the study, discussing construct validity regarding the taxonomy, internal validity regarding model scale, and external validity regarding domain generalization.
- **Chapter 8:** Conclusions summarizes the work undertaken, discusses the implications for the software engineering community, and outlines future research directions, such as IDE integration and cross-domain generalization.

CHAPTER 2

State of the Art

2.1 Prompt Engineering

Prompt Engineering currently represents the reference methodology for input optimization in Large Language Models (LLMs). According to the definition proposed by Singh et al. [9], prompt engineering is a rapidly evolving discipline focused on optimizing the interaction between humans and Large Language Models (LLMs). In its essence, it entails the design and structuring of inputs, known as prompts, in order to maximize the accuracy and relevance of the responses generated by Artificial Intelligence systems.

Therefore, the primary objective of this discipline is not limited to the simple formulation of requests, but extends to the creation of structured guidelines that allow the model to operate within specific constraints. The quality of the prompt is decisive: well-engineered inputs unlock complex reasoning capabilities, while ambiguous instructions significantly degrade the model's performance.

2.1.1 Historical Evolution

The approach to Natural Language Processing (NLP) tasks has undergone a radical transformation over time. Initially, each problem required dedicated architectures trained from scratch; subsequently, with the advent of Transformer-based models, such as BERT (Devlin et al. [10]), the **Transfer Learning** paradigm took hold, where a generic model was specialized for individual tasks through supervised fine-tuning.

A decisive breakthrough came with the **text-to-text** approach, introduced by models like T5 (Raffel et al. [11]), which made it possible to unify various linguistic problems under this single framework by reformulating every linguistic task as a text generation assignment.

However, it is with the advent and expansion of **Generative Pre-trained Transformers** (GPT, particularly GPT-3 and its successors) that Prompt Engineering has assumed a central role. As demonstrated by Brown et al. [1] in their work, these models, thanks to their massive parametric scale, possess **In-Context Learning** capabilities. This means they can solve new tasks simply by receiving adequate instructions or examples within the prompt, without the need to update the model’s neural weights. This shift from “fine-tuning” to “prompting” marks the origin of the current era of AI interaction.

2.1.2 The Shift to Semantic Interaction

The widespread diffusion of consumer applications based on Large Language Models (LLMs), ranging from direct interaction tools like ChatGPT or Gemini to integrated AI assistants, has fundamentally altered the landscape of problem-solving. Literature across diverse domains, from medicine to mechanics, documents a paradigm shift in how professional tasks are executed [12]. Some examples of this literature are Maity et al. [13], which examines the use of LLMs in medical applications related to diagnosis support, and Liu et al. [14], which studies the reliability of these tools in the structural analysis of buildings. This transition marks a move away from rigid, syntax-based command execution toward a model of semantic interaction, where the user’s intent acts as the primary driver of computational output. However, the efficacy of this interaction is not guaranteed. In a landscape dominated by these

technologies, the ability to orchestrate them effectively becomes a determinant of success. A well-engineered input can unlock complex reasoning capabilities, whereas ambiguous instructions significantly degrade the model's performance. Tripathi et al. [15] define this as the **Instruction Gap**, highlighting that while LLMs excel at general tasks, they frequently fail to adhere to precise enterprise-level constraints without rigorous instruction tuning. Consequently, the user's role evolves into that of a 'semantic architect', tasked with designing prompts that bridge the gap between abstract human intent and the model's execution, a challenge highlighted by Zhou et al. [16], who note that LLMs function as 'black-box computers' where plain language does not always produce the desired results due to non-intuitive processing.

2.1.3 Impact on Software Development

This shift is particularly evident in the software development lifecycle. LLMs are widely used throughout the software development phases, especially in code generation. This has led to the creation of specialized software development assistants, such as GitHub Copilot, and their subsequent direct integration within **IDEs** (Integrated Development Environments) to provide real-time support to developers.

As a result, software creation is no longer exclusively defined by writing syntactic rules but is evolving toward a hybrid paradigm that relies heavily on semantic interaction with the model.

Recent empirical studies corroborate this massive adoption. The work by Della Porta et al. [17], highlights that developers extensively use LLMs like ChatGPT for diverse coding tasks. However, the study notes that while adoption is high, model performance remains inconsistent and vulnerable to **hallucinations**. Because outputs cannot be assumed to be accurate, users must rigorously validate results. This dependency implies that developer productivity is no longer solely a function of coding skill, but is increasingly linked to their ability to formulate effective prompts that constrain the model to specific business objectives. In this environment, the ability to describe functional requirements to the LLM in a formal and explicit manner becomes critical. An imprecise description or a poorly structured prompt can lead to the generation of code that is syntactically correct but functionally incorrect.

2.1.4 The Increasing Value of the Prompt

As the barrier to accessing LLMs lowers, the strategic value shifts from the model itself to the Prompt. Prompt Engineering, therefore, transcends simple query formulation; it represents a rapidly evolving discipline focused on optimizing the interaction between humans and LLMs. As defined by White et al. [18], prompts are a form of programming that customizes the outputs and interactions with an LLM. Consequently, the prompt becomes the formal encoding of the user’s intent, a **textual artifact** that must encapsulate the request, the context, and the constraints in a format the model can unambiguously interpret. This discipline serves as the semantic bridge between the user’s functional requirements and the computational capacity of the LLM. Just as traditional engineering requires blueprints to ensure structural integrity, effective interaction with Generative AI requires prompts to be treated as **engineered specifications**. Without this rigor, the interaction remains a process of *trial and error*, preventing the systematic optimization and reliability required in professional software development. This necessity for rigor underscores the urgent need for a structured representation of prompts.

2.2 Prompt Engineering Fundamentals

To construct an effective prompt, it’s crucial to understand its structure and analyze its **syntactic**, **semantic**, and **logical** aspects. While early interactions with LLMs were unstructured, the literature has since converged on a formal definition of the prompt’s constituent parts.

According to the comprehensive taxonomy by Schulhoff et al. [19], a prompt is not a monolithic string but a composite artifact. While not all elements are mandatory for every query, Schulhoff et al. [19] identify a standard anatomy comprising specific key components whose explicit separation is vital for performance:

- **Instruction (or Directive):** This constitutes the core of the prompt. Schulhoff et al. [19] define it as the explicit statement describing the task the model should perform (e.g., “Summarize”, “Translate”, “Classify”).

- **Context:** A section that provides background information, external knowledge, or the specific *Persona* the model should adopt. This component semantically guides the response, influencing tone and domain specificity. As noted by White et al. [18], context often acts as a global constraint system (e.g., “You are a Senior Java Developer”), narrowing the search space of the model’s probability distribution.
- **Input Data:** The specific content or variables the model is required to process (e.g., the code snippet to be debugged or the text to be summarized). Schulhoff et al. [19] distinguish this as the “Content” or “Example” slot, noting that separating the *data* from the *instruction* is a fundamental principle of robustness against prompt injection.
- **Output Indicator:** The explicit definition of the desired output format (e.g., “JSON format”, “Bullet points”, or specific code conventions). Specifying the output format reduces the need for post-processing of the model’s response and aligns the generation with downstream application requirements.

In an optimal prompt, these components are not just present but are orchestrated through three engineering dimensions:

- **The Syntactic Aspect:** This concerns the structural formatting of the prompt. Chen et al. [20] argue that because natural language is inherently ambiguous, “Promptware” requires the use of clear delimiters (such as XML tags, quotes, or Markdown headers) to syntactically separate instructions from data. This syntax facilitates the model’s ability to parse the input tokens correctly.
- **The Semantic Aspect:** This concerns word choice and meaning. To ensure the prompt is “executable” by the model, developers must avoid vague or ambivalent terms. The goal is to maximize Semantic Alignment, ensuring the user’s mental model matches the model’s interpretation of the vocabulary [20].
- **The Logical Aspect:** This concerns the internal coherence and flow of reasoning. For complex tasks, Cheng et al. [21] demonstrate that a monolithic prompt often fails; instead, the logical aspect involves breaking down the request into ordered

sub-instructions (Chain-of-Thought), explicitly guiding the model through the reasoning steps required to reach the solution.

2.2.1 Overview of Current Paradigms

To maximize the inferential capabilities of LLMs, the research community has developed a comprehensive taxonomy of prompting techniques, as described by Sahoo et al. [22].

These techniques range from **Zero-Shot Prompting**, where the model relies entirely on prior knowledge without explicit training data, to **Few-Shot Prompting**, which leverages **In-Context Learning** by providing input-output examples to guide the model's performance.

More advanced paradigms have been introduced to handle complex reasoning tasks. **Chain-of-Thought** (CoT) transforms the model into a sequential reasoner by inducing intermediate logical steps, bridging the gap between complex logic and final answers. Extensions of this, such as **Tree-of-Thoughts** (ToT), allow the systematic exploration of multiple reasoning paths by maintaining a tree structure of intermediate "thoughts". Other methods, such as **Role Playing** or **emotion-based prompting**, constrain the model's perspective or emotional tone to enhance applicability in specific domains. Collectively, these techniques demonstrate that the structure of the prompt is a decisive factor in determining the quality of the model's reasoning.

2.2.2 The Gap Between Theory and Practice

Despite the theoretical effectiveness of these advanced patterns, there is a significant disconnect in practical application. In their empirical assessment of ChatGPT-generated code, Della Porta et al. [17] analyzed over 12,000 user prompts and found a strong preference for *Zero-Shot Prompting* (10,034 occurrences) compared to *Few-Shot* (576 occurrences) or *Zero-Shot with CoT* (713 occurrences).

Based on the findings reported by Della Porta et al. [17], the insights regarding prompt usage and adoption can be summarized as follows:

- **Preference for Simplicity:** Developers overwhelmingly gravitate toward Zero-Shot prompting due to its simplicity, low barrier to entry, and minimal setup

requirements. This preference aligns with the efficiency demands of agile software development workflows, allowing for rapid prototyping without extensive customization.

- **Barriers to Advanced Patterns:** The cognitive load and additional effort required to manually structure complex prompts act as a significant barrier to adoption. Techniques such as Chain-of-Thought (CoT) or detailed Personas require a higher level of understanding and structuring to be effective, which discourages their use despite their potential benefits.
- **Underutilization of Techniques:** Consequently, a critical gap exists where advanced prompting techniques are underutilized. While these methods could offer enhanced results for complex reasoning tasks, they are frequently neglected because implementing them manually in unstructured text is perceived as difficult and time-consuming compared to the immediate usability of simple prompts

2.3 The Engineering Gap in Prompt Development

While the taxonomy of prompting techniques provides powerful heuristics for enhancing model performance, their practical implementation remains fraught with instability. Current literature [23, 20, 18] identifies a growing *reproducibility crisis* in prompt engineering, where the lack of systematic standards leads to fragile and unpredictable system behaviors.

2.3.1 The Fragility of Unstructured Prompts

A major limitation in current workflows is the **epistemic fragility** of Large Language Models (LLMs). Research by Krastev et al. [23] demonstrates that prompt framing systematically modulates the model’s ability to correct misinformation, often overriding its internal knowledge. Specifically, factors such as adopting an **assertive expert role**, expressing **creative intent**, or using **epistemically closed framing** can significantly reduce a model’s willingness to refute false claims.

This sensitivity transforms prompt development into a stochastic process, as models may prioritize conversational compliance over epistemic integrity. Unlike rigid syntax in traditional software, prompts function as ‘soft’ code where even trivial edits in tone or context can lead to inconsistent outputs, undermining the reliability required for robust engineering.

2.3.2 The Limitations of “Trial and Error”

The predominant methodology for prompt optimization remains ad-hoc trial and error. Chen et al. [20] characterize this current state as the *promptware crisis*, noting that unlike traditional software development, which relies on formal languages and deterministic environments, prompt development is currently based on ambiguous, unstructured natural language. Consequently, developers are forced to rely on time-consuming, manual iteration rather than rigorous engineering principles.

White et al. [18] further highlight that without a structured approach or recognized patterns, developers often depend on intuition to solve common problems, lacking the reusable **knowledge transfer** mechanisms found in standard software engineering. This manual “fuzzing” is particularly detrimental when scaling to complex tasks. Cheng et al. [21] demonstrate that unstructured text is inherently limited for multi-step reasoning.

Natural Language often encompasses complex relationships that are difficult to maintain in a linear format. They argue that without an explicit structure (such as a graph) to guide the model, LLMs struggle to navigate long-term dependencies. Thus, the transition from simple **Zero-Shot** interactions to complex reasoning structures exacerbates the issue, as the reliance on unstructured text exponentially increases the surface area for semantic errors and reduces reproducibility.

2.3.3 The Need for Systematic Engineering

To mitigate these risks, the field is moving towards **Promptware Engineering**, a paradigm formally proposed by Chen et al. [20] to bridge the gap between ad-hoc prompting and formal software engineering principles. This shift necessitates treating prompts not just as text strings, but as *Promptware*: first-class engineered artifacts

with defined schemas, constraints, and version control.

This transition is critical because, as highlighted in the systematic survey by Sahoo et al. [22], the current lack of organizational standards leads to a chaotic landscape of techniques. This disorganization results in the accumulation of **Prompt Debt**, technical debt arising from poorly structured and inefficient prompts, which hinders long-term application agility. Therefore, a formal representation is required to stabilize these techniques, ensuring that the advanced reasoning patterns discussed in Section 2.2 can be deployed reliably within a rigorous engineering framework.

2.4 The Unstructured Data Problem

2.4.1 The Absence of Structural Representation

The fundamental limitation hindering the *Promptware* paradigm is the lack of a formal Data Structure to represent prompts. While concepts like “Promptware” treat prompts as engineering artifacts, the underlying representation remains a flat, unstructured sequence of characters. This creates a disconnect between the conceptual model of a prompt (which has distinct logical parts) and its physical representation (a raw string).

This *string-based* paradigm makes component addressability impossible. In strict software engineering, objects have accessible properties (e.g., `Class.Method()`). In contrast, a prompt string offers no native mechanism to isolate or target specific semantic components, such as the **Context** or the **Constraints**, without fragile string manipulation.

To overcome these limitations, recent literature has begun attempting to impose structural scaffolding on prompts. Cheng et al. [21] demonstrated that the linear nature of text is insufficient for complex tasks, proposing a **Structure Guided Prompt** framework that converts text into a **graph-based structure**. By explicitly mapping entities and relationships, they proved that structural navigation significantly outperforms linear string processing for multi-step reasoning. Similarly, Han et al. [24] introduced **Prompt Tuning with Rules** (PTR), which addresses the need for compositionality. They argue that complex classification tasks cannot be solved by a

monolithic string but require constructing prompts from discrete sub-prompts (logic rules) that interact hierarchically.

Parallel efforts focus on abstracting the prompt into a programmable interface. Khattab et al. [25] propose **DSPy**, moving entirely away from manual string editing towards defining prompts as “signatures” (input/output type definitions) within a self-optimizing pipeline. Complementing this, Xing et al. [26] introduce **CNL-P**, arguing that robust human-AI interaction requires a *Controlled Natural Language* with strict syntax, effectively treating the prompt as an API with typed inputs rather than a free-text box.

However, while these works validate the necessity of structure, a gap remains. Cheng et al. [21] focus on reasoning graphs for the model; Khattab et al. [25] focus on abstraction layers to hide the prompt. There is currently no standardized **general-purpose data structure**, akin to a Document Object Model (DOM) or Abstract Syntax Tree (AST), designed specifically to represent the anatomy of a raw prompt. The lack of such a representation prevents the development of agnostic tools capable of parsing, traversing, and validating the internal components of a prompt before it is sent to a model.

2.4.2 The Reproducibility Crisis

Because prompts are currently designed as ad-hoc strings without a formalized structure, they are difficult to analyze, version, and report. Current methodologies often rely on subjective “trial and error”, where the success of a prompt depends on hidden variables rather than engineered principles.

In scientific and engineering contexts, this creates a significant reproducibility crisis. Sallou et al. [27] identify this as a critical “Threat to Validity” in their systematic review of LLM-based software engineering. They found that many studies suffer from “reproducibility smells”, such as the failure to report the full prompt content, the specific system instructions, or the model hyperparameters. Because the prompt is treated as a simple string rather than a structured artifact, researchers often omit the specific “context” or “constraints” used during experimentation.

This lack of “Descriptive Validity” renders experimental results ephemeral. As

Chen et al. [20] argue in their definition of the “Promptware Crisis”, the absence of a rigorous format means that a “working” prompt is often just a result of overfitting to a specific session or model version. Without a standardized data structure to encapsulate and separate the instruction, context, and data, it is impossible to guarantee that a prompt will perform consistently across different environments, preventing prompt engineering from maturing into a reliable engineering discipline.

2.4.3 Lack of Granular Control

To perform advanced prompt engineering tasks, it is essential to be able to identify and manipulate specific components of a prompt. For instance, following the “Persona Pattern” described by White et al. [18], a developer might want to swap the “Role” from Software Architect to Junior Developer to observe changes in code quality, while keeping the “Main Instruction” identical.

In the current unstructured paradigm, performing this task is difficult and error-prone. Zamfirescu-Pereira et al. [28] highlight that users engage in “superstitious” and “opportunistic” behavior due to overgeneralization based on single successes and the application of human social mental models to AI, which leads them to avoid systematic iterations on prompts. Although visual tools like ChainForge (Arawjo et al. [29]) attempt to introduce modularity, they are often limited by the underlying textual representation of the prompt, which does not natively support object-oriented manipulation. Sahoo et al. [22] further emphasize that advanced techniques like “Prompt Tuning” require distinct handling of task-specific vectors versus fixed instructions. The absence of a standard data structure (akin to a DOM for HTML) means standard workflows lack the tooling to decompose a prompt into these constituent parts automatically, slowing down the development cycle and increasing the risk of regression during optimization.

Our Contributions

To bridge the engineering gap between unstructured prompt design and rigorous software practices, this thesis introduces the **Prompt Syntax Tree (PST)**, a formal hierarchical representation inspired by compiler theory that decomposes prompts into addressable semantic units. This framework is operationalized through a **PST-Aligned Dataset**—a novel supervised corpus of human-annotated prompts mapped to JSON structures—and an **Automatic Parser**, a fine-tuned language model capable of performing lossless Prompt-to-PST translation for systematic validation and analysis.

The Prompt Syntax Tree (PST)

3.1 Introduction

Starting from the literature review in *Chapter 2* (specifically Section 2.4.1), the research community has identified the “unstructured string” representation of prompts as a primary bottleneck in prompt engineering. Consequently, recent efforts have focused on imposing structural constraints on prompts to improve their reliability and control.

However, existing solutions have been largely ad hoc and domain-specific. For example, Cheng et al. [21] proposed a *Structure-Driven Prompt* specifically for mapping entities in a graph for reasoning tasks, while Khattab et al. [25] introduced abstract signatures in **DSPy** to optimize pipeline interactions. While effective in their respective domains, these approaches lack generality: they do not provide a universal representation capable of describing any generic prompt.

This chapter proposes a unified solution: the **Prompt Syntax Tree** (PST). Drawing an analogy with the **Abstract Syntax Tree** (AST) used in compiler theory, which provides a hierarchical representation of source code regardless of specific logic, the PST aims to provide a formal and generalized data structure for prompts. To define the necessary components of this structure, we first analyze the established taxonomy

of prompting techniques defined by Sahoo et al. [22] to identify the fundamental building blocks (semantics, logic, and syntax) that the PST must represent. Next, we formalize the PST schema and define its constituent nodes.

3.2 Prompt Taxonomy

As described by Sahoo et al. [22], there are several prompting strategies that allow for the maximization of the inferential capabilities of LLMs. The most relevant techniques are analyzed below.

3.2.1 Zero-Shot Prompting

This technique requires the model to be provided only with a description of the task to be performed, without any demonstrative examples (input/output). Therefore, in the prompt, the model receives only the description of the task to be performed, without any other data to rely on. The model must therefore rely entirely on its prior knowledge and generalization capabilities to complete the task. The Figure 3.1 shows an example of this type of prompt



Figure 3.1: Example of zero-shot prompt from the site promptingguide.ai

This approach was formalized in the paper Radford et al. [30] where the authors tested the models' ability (GPT-2) to perform tasks based solely on natural language instructions. This demonstrated that the models not only memorize the vocabulary of the text corpora on which they were trained, but also learn the logic needed to manipulate symbols in tasks such as translation and summarization, modeling the conditional distribution of the text during pre-training.

3.2.2 Few-Shot Prompting

Few-Shot Prompting involves providing the model with a set of examples (input/output pairs) within context, before the actual request, to help it understand a given task. This technique leverages the LLM’s **pattern-matching** capabilities by forcing it to imitate the logic, tone, format, or style demonstrated in the examples without updating its weights. In practice, this involves “temporary fine-tuning” the model for the duration of a single session, providing a set of N examples (usually between 3 and 10) from which the model can infer the *latent rules* connecting input to output. This mechanism is also known as **In-Context Learning**. Precisely because of how this type of prompt works, the selection and composition of prompt examples are crucial, as they can significantly influence the model’s behavior. Correct management of this type of prompt is therefore essential to achieve optimal performance and mitigate bias.

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

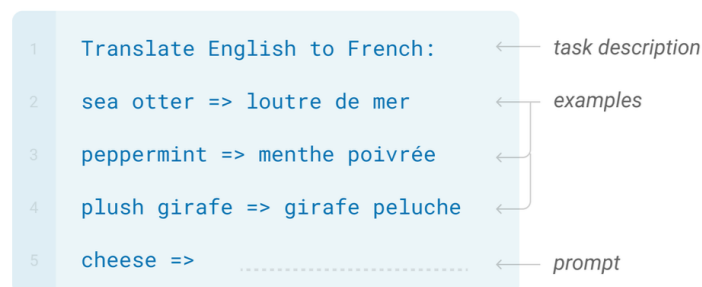


Figure 3.2: Example of few-shot prompt from Brown et al. [1]

This type of prompting and the formalization of the concept of in-context learning were introduced in the paper Brown et al. [1], with GPT-3. Specifically, the paper defines two types of approaches for providing task examples to the model:

- **Few-Shot:** shown in Figure 3.2, several task demonstrations are provided. K context examples are provided, each with the corresponding task completion, followed by a single context example, for which the model is expected to provide the solution.

- **One-Shot:** shown in Figure 3.3, a single task demonstration is provided. This distinction mirrors the way tasks are assigned to humans, where a single example is often sufficient to clarify the objective.

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.



Figure 3.3: Example of one-shot prompt from Brown et al. [1]

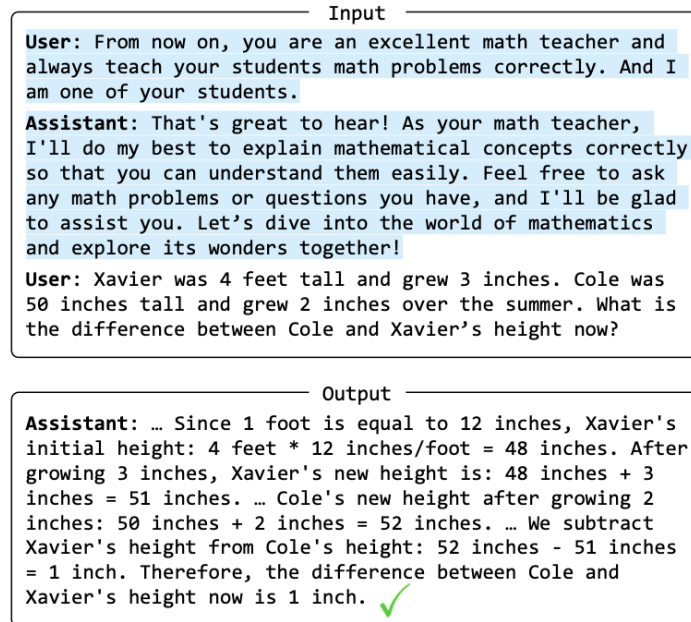
3.2.3 Role Playing Prompting

Role Playing instructs the model to adopt a specific identity, profession, or role before executing the task. The underlying mechanism is defined as **Contextual Anchoring**. When an LLM processes a generic request, it relies on its general knowledge to provide an answer, thereby drawing from the entirety of the data on which it was trained. By assigning a role, however, the model is constrained to prioritize a specific subset of data and linguistic patterns pertinent to that role. Formalized by Kong et al. [2], this technique acts as a context switch: by shifting the model’s focus to a specific knowledge domain, it enhances its reasoning capabilities and the sector-specific relevance of the responses. An example of this technique is shown in Figure 3.4. This is made possible by the origin of the model’s training data. According to the authors, this technique is effective because an LLM’s training data originates from diverse sources. By assigning a role, the model is able to focus on a specific subset of data associated with the defined role, rather than performing generic text generation.

3.2.4 Chain-of-Thought (CoT) Prompting

One of the main limitations of Large Language Models (LLMs) emerges when they are confronted with tasks requiring complex reasoning. Classical prompting

(b) Role-Play Prompting

**Figure 3.4:** Example of role playing prompt from Kong et al. [2]

methods treat the model as a **pattern matcher**, meaning the model is expected to map the input x directly to the final output y ($x \rightarrow y$), relying exclusively on the probabilities learned during pre-training. This approach, shown in figure 3.5, is likened to “*System 1 thinking*” (fast, intuitive, and automatic), which, however, fails when sequential logical deliberation is necessary. To overcome this obstacle, Wei et al. [3] introduced the **Chain-of-Thought (CoT)** paradigm. In this framework, the model is treated as a **Sequential Reasoner**. Instead of jumping directly to the final conclusion to provide the task answer, CoT induces the generation of a series of **intermediate reasoning** steps that logically connect the input to the final response. From a computational standpoint, generating these intermediate steps (of “thought”) allows the model to allocate more computing power (and more tokens) to the resolution of the problem. This process transforms the direct mapping ($input \rightarrow output$) into a structured sequence $x \rightarrow z \rightarrow y$, where z represents the chain of thought, providing the model with a structure to refer to when generating the final response. The authors demonstrated that providing the model with **few-shot** examples that include the reasoning steps necessary to reach the final solution (<Input, Chain of Thought, Output>) leads to a massive increase in model performance, allowing it to solve

problems previously considered impossible.

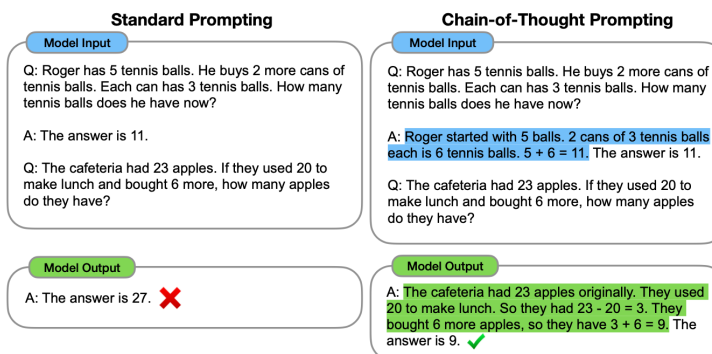


Figure 3.5: Example of Chain-of-Thought prompt from Wei et al. [3]

3.2.5 Zero-Shot Chain-of-Thought Prompting

Introduced by Kojima et al. [4], Zero-Shot Chain-of-Thought represents an important variation of the CoT paradigm, challenging the assumption that LLMs require specific reasoning examples to activate their reasoning capabilities. While classical CoT relies on imitating examples provided by the user, Zero-Shot CoT demonstrates that **multi-step reasoning** capabilities are intrinsic to the model and can be unlocked through a simple *trigger phrase*. The innovation, therefore, lies in adding an explicit constraint to the original query: a phrase that instructs the model not to optimize immediately for the final answer tokens (i.e., the optimal tokens based on the user’s request), but rather for a reasoning sequence. From the scores of the various sentences tested as trigger phrases, shown in figure 3.6, it can be seen that the most effective trigger phrase identified by the authors is: “Let’s think step by step”.

This simple instruction acts as a **control code** that modifies the output probability distribution. Instead of directly generating the solution, the model is forced to generate a sequential logical structure. Technically, the process described by Kojima et al. is articulated in two distinct phases:

- Reasoning Extraction: Concerns the generation of reasoning. The trigger phrase T is added to the original prompt P , through which the model generates a chain of thought by “talking to itself” to decompose the problem.

No.	Category	Template	Accuracy
1	instructive	Let's think step by step.	78.7
2		First, (*1)	77.3
3		Let's think about this logically.	74.5
4		Let's solve this problem by splitting it into steps. (*2)	72.2
5		Let's be realistic and think step by step.	70.8
6		Let's think like a detective step by step.	70.3
7		Let's think	57.5
8		Before we dive into the answer,	55.7
9		The answer is after the proof.	45.7
10	misleading	Don't think. Just feel.	18.8
11		Let's think step by step but reach an incorrect answer.	18.7
12		Let's count the number of "a" in the question.	16.7
13		By using the fact that the earth is round,	9.3
14	irrelevant	By the way, I found a good restaurant nearby.	17.5
15		AbraKadabra!	15.5
16		It's a beautiful day.	13.1
-		(Zero-shot)	17.7

Figure 3.6: trigger phrase selection from Kojima et al. [4]

- **Answer Extraction:** Concerns the extraction of the answer. A second prompt is concatenated to the sequence generated in the first stage, forcing the model to converge, based on the reasoning just produced, toward a conclusive and correctly formatted answer.

The discovery of this mechanism, an example of which is shown in Figure 3.7, occurred thanks to an in-depth **ablation study** conducted by the authors. By testing various syntactic formulations, it emerged that the explicit instruction for sequentiality is crucial, as it shifts the model's attention toward producing intermediate steps instead of jumping directly to the final conclusion. This bridges the performance gap between zero-shot methods and the more costly few-shot methods, without the need to engineer manual examples for every specific task.

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓*

Figure 3.7: Example of Zero-Shot Chain-of-Thought prompt from Kojima et al. [4]

3.2.6 Tree-of-Thoughts (ToT) Prompting

The **Tree-of-Thoughts (ToT)** framework, introduced by Yao et al. [5], represents a structural evolution of previous methods, generalizing the linear concept of Chain-of-Thought. While CoT prompting constrains the model to follow a single sequential path of thought ($x \rightarrow z \rightarrow y$), ToT enables the exploration of multiple simultaneous reasoning paths, treating the problem-solving process as a search on a tree, as shown in Figure 3.8.

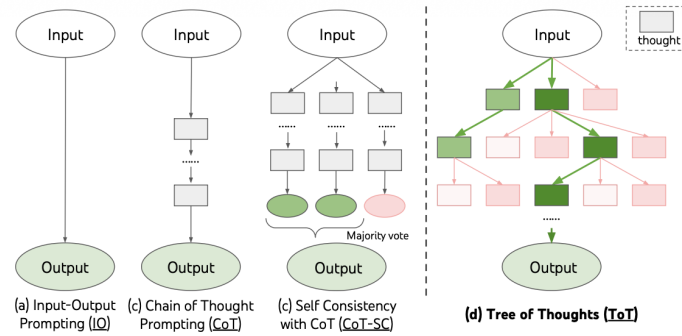


Figure 3.8: Tree-of-Thoughts representation from Yao et al. [5]

The core idea is that complex reasoning does not occur in a single pass, instead, it requires planning, the exploration of alternatives, and **backtracking** capabilities. In this methodology, each node of the tree is represented by a partial state of the solution, defined as a *thought*, a coherent text sequence that serves as an intermediate step toward the final solution. ToT is based on four functional modules that interact with each other:

- **Decomposition:** The problem is broken down into simpler intermediate steps.
- **Thought Generator:** Starting from one of these thoughts (called “states” of the tree), the model generates k possible candidates for the next reasoning step.
- **State Evaluation:** The model (or an external component) evaluates the progress of each branch toward the solution by assigning a score. This allows for the pruning of unproductive branches.
- **Search Algorithm:** An external search controller navigates the tree using classic search algorithms such as BFS (Breadth-First Search) or DFS (Depth-First

Search).

From an implementation perspective, ToT utilizes a **hybrid approach**. It relies not only on textual prompting (System 1) but also integrates an algorithmic controller (System 2), often an external Python script, which manages the tree data structure and iteratively invokes the LLM to generate or evaluate nodes. However, it is still possible to emulate this prompting framework using a single prompt that forces the model to simulate the deliberation process. Figure 3.9 shows an example of this type of implementation.

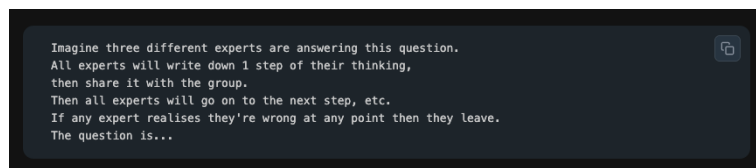


Figure 3.9: Tree-of-Thoughts representation from the site promptingguide.ai

3.2.7 Graph-of-Thought (GoT) Prompting

Overcoming the hierarchical constraints imposed by the tree structure of Tree-of-Thoughts, Besta et al. [6] introduce the **Graph-of-Thoughts (GoT)** paradigm. This framework models the LLM's reasoning process no longer as a tree, but as an arbitrary **Directed Acyclic Graph (DAG)**. The fundamental difference compared to previous methods lies in **topological flexibility**: while CoT is a line and ToT is a branching tree, GoT allows reasoning paths to split but also to converge, as shown in Figure 3.10. This enables complex cognitive operations inaccessible to other methods, such as aggregation (combining multiple promising thoughts into a new, stronger solution) and the recirculation of information between distant branches.

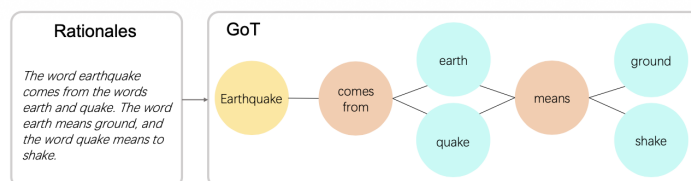


Figure 3.10: Graph of Thoughts deduction example from Besta et al. [6]

In GoT, every **vertex** of the graph represents an “information unit” (a thought, a paragraph, an intermediate result), while the **edges** represent the logical dependencies between them. This structure is managed by a modular architecture composed of four main components:

- **Prompter:** The module responsible for translating the graph state into a textual prompt understandable by the LLM. It handles encoding the graph structure and previous thoughts into the context.
- **Parser:** Extracts relevant information from the LLM’s raw response and updates the internal graph state, creating new nodes (thoughts) from the generated output.
- **Scoring:** Evaluates the quality of vertices (thoughts) by assigning them a score. This allows the system to determine which lines of reasoning are promising and which must be discarded.
- **Controller:** This is the “brain” of the framework, orchestrating the entire reasoning process. It selects which topological operations to apply to existing thoughts. The fundamental operations defined by Besta et al. include:
 - **Generation:** Creation of a new thought based on an existing one.
 - **Aggregation:** Fusion of multiple thoughts into a single one, to create a synthesis or combine the strengths of different solutions.
 - **Refinement:** Iterative improvement of a single thought (loop).

With GoT, reasoning is no longer a purely textual flow generated one-shot by the model, but an **algorithmically constructed process** where the LLM acts as a processing engine for individual nodes, while the overall logic is maintained by the graph structure. However, it is still possible to simulate these operations by adding a trigger phrase to a prompt, such as “*think about this from multiple angles and combine the best ideas*” (as is done for ToT or zero-shot CoT).

3.2.8 Self-Refine Prompting

Self-Refine Prompting, introduced by Madaan et al. [7], represents a paradigm shift compared to one-shot generation. Instead of attempting to obtain the optimal answer in a single pass, this framework implements an **iterative refinement mechanism** entirely managed by the model itself, without the need for external supervision or labeled training data. The **asymmetry between the generative and discriminative capabilities** of LLMs lies at the basis of this method, models tend to be better critics than they are creators. It is often easier for a model to recognize an error in an existing output (even its own) than to generate a perfect output from scratch. The framework operates through a continuous cycle composed of three distinct phases:

- **Initial Generation:** The model produces a first draft of the response to the given input, using a standard prompt.
- **Feedback:** The model is instructed, via a second prompt, to review its own output. In this phase, the model acts as a critic, identifying discrepancies, logical errors, or shortcomings regarding the task requirements, generating structured feedback.
- **Refinement:** The model receives its original draft and the newly generated feedback as input. Based on these, it produces an improved version of the response.

This cycle is repeated for a predefined number of iterations or until a **stopping condition** is met (e.g., when the feedback no longer detects significant errors). Unlike **Reinforcement Learning from Human Feedback (RLHF)** methods which require expensive weight training, Self-Refine is an **inference-time optimization** technique. It emulates the human cognitive process of draft revision, allowing the model to **self-correct** in domains ranging from code generation to mathematical problem solving, improving the model's average performance simply by extending the duration of the interaction. An example of this type of prompt is shown in Figure 3.11

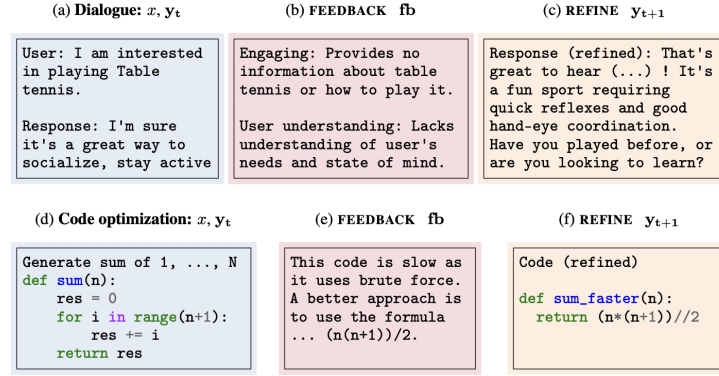


Figure 3.11: Self-Refine prompting example from Madaan et al. [7]

3.2.9 Emotion Prompting

Emotion Prompting, proposed by Li et al. [8], explores a different dimension of human-machine interaction: the use of emotional stimuli to modulate the performance of language models. This technique is grounded in the hypothesis that LLMs, having been trained on vast corpora of text produced by humans, have encoded not only semantic and syntactic knowledge but also a latent form of “*emotional intelligence*” and sensitivity to social context. While standard prompting methods formulate a request as neutral information retrieval, Emotion Prompting reformulates the interaction as a high-priority social scenario. The underlying mechanism is probabilistic in nature: in the pre-training dataset, words such as “*urgent*”, “*critical*”, “*danger*”, or references to personal consequences (“*important for my career*”) are statistically correlated with human responses characterized by greater care, precision, and analytical depth. By injecting these emotional stimuli into the prompt, the model’s **probability distribution** is manipulated, shifting the focus of attention toward tokens that compose higher-quality outputs. This effectively simulates a “*high-effort*” response typical of a human under pressure. The authors, drawing inspiration from cognitive psychology theories such as **Social Cognitive Theory** and **Self-Monitoring Theory**, codified 11 specific templates (e.g., “This is very important to my career”, “You’d better be sure”). Experiments demonstrate that adding these phrases, which function as emotional constraints rather than logical ones, leads to measurable improvements in performance on complex tasks. This suggests that the reasoning capacity of LLMs can be influenced by factors that simulate emotional

states. Figure 3.12 shows the difference in performance on different models obtained using Emotion Prompt compared to a normal prompt.

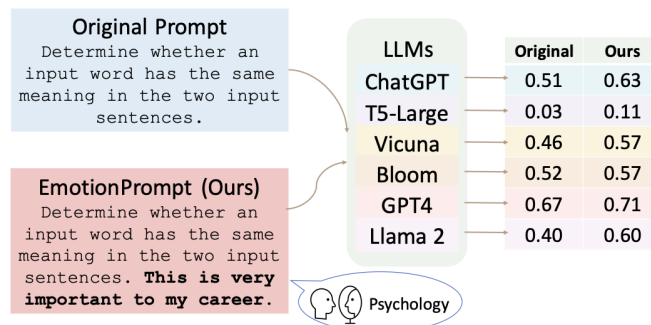


Figure 3.12: Emotion Prompting example from Li et al. [8]

The techniques analyzed in previous sections (CoT, ToT, GoT, etc.) focus primarily on optimizing the model’s intrinsic reasoning process, leveraging the **In-Context Learning** paradigm. However, the prompt engineering landscape is vast and includes **orthogonal approaches** that address aspects beyond pure logical sequentiality. A relevant class of methods concerns modifying the **representation of the problem**. Techniques such as **Chain-of-Table** Wang et al. [31], for instance, compel the model to structure intermediate data in a tabular format, facilitating inference on semi-structured information. Parallely, approaches like **Program-of-Thoughts (PoT)** or **Code Prompting** reformulate reasoning not in natural language, but through **executable code** (e.g., Python), leveraging the logical rigor learned by models on programming datasets to solve mathematical or algorithmic tasks. Finally, a fundamental frontier is represented by the integration of **external modules** to overcome the limitations of the static knowledge contained within the model’s weights. The most prominent example is **Retrieval-Augmented Generation (RAG)** Lewis et al. [32]. In this framework, the prompt is no longer an isolated entity but the result of a **dynamic process**: a query is used to retrieve relevant documents from an external knowledge base, which are then injected into the model’s context. This hybrid approach not only improves factual accuracy but significantly mitigates the phenomenon of **hallucinations**, anchoring generation to verified sources. In summary, the evolution of prompting is transitioning from simple query formulation (System 1) to the design

of complex cognitive architectures (System 2) that orchestrate reasoning, structured formats, and external tools.

3.3 Formalizing the Prompt Structure

3.3.1 From Linear String to Hierarchical Representation

The transition from a linear string to a structural representation is not arbitrary; it stems directly from the convergence of the “Prompt Fundamentals” analyzed in *Chapter 2* and the advanced taxonomies discussed in the previous section. As established by Schulhoff et al. [19], a prompt is intrinsically composed of semantic blocks: **Instruction**, **Context**, **Input Data**, and **Output Indicators**. However, while this anatomy provides a foundational vocabulary, it treats these components as flat, independent segments. Our analysis of advanced paradigms, such as *Chain-of-Thought* and *Tree-of-Thoughts*, reveals that this flat model is insufficient for complex engineering tasks.

Therefore, the PST proposal represents a refinement and evolution of Schulhoff’s model. We reorganize the standard components into a structured hierarchy (e.g., subsuming Data and Output under *Context*) and extend the schema with new branches dedicated to *Reasoning* and *Examples*. Formally, this allows us to move from a linear sequence of characters $P = \{c_1, c_2, \dots, c_n\}$, where components are implicit substrings $P[i : j]$, to a structured object T . Let T be a tree with root $T = (N, E)$, where:

- N is the set of nodes representing the specific semantic units derived from our synthesis of Schulhoff’s anatomy and advanced prompting techniques (e.g., *Role*, *Constraint*, *Influence*).
- E is the set of edges representing the hierarchical relationships between these units.

In this formalization, the raw text string is no longer the primary object, but rather the final result of traversing the tree. This structural definition provides the theoretical foundation for the Prompt Syntax Tree (PST).

3.3.2 The AST Analogy

To operationalize this hierarchical definition, this thesis adopts the conceptual framework of the **Abstract Syntax Tree** (AST) used in compiler theory.

Just as an AST provides a tree-like representation of the syntactic structure of source code, where nodes denote a specific construct occurring within the code, the PST provides a hierarchical tree-like representation of a natural language prompt.

Within this structure, the **root** represents the prompt in its entirety, while the various branches represent specific semantic sections (such as context, main statement, etc.). This approach allows a linear text string to be transformed into a structured format, facilitating automatic decomposition and computational analysis.

3.4 Derivation of the PST Components

The specific design of the nodes within the PST is the result of the harmonization process described above: combining the foundational components identified by Schulhoff et al. [19] with the structural requirements of the advanced taxonomies [22] (Zero-Shot, CoT, ToT).

This synthesis led to the identification of four macro-categories that constitute the first level of the tree: **Main Instruction**, **Context**, **Examples**, and **Reasoning**. The derivation and function of each are detailed below.

3.4.1 Core Components

Main Instruction

Every prompt, regardless of its complexity, contains a central directive. This maps directly to the **Instruction** component defined by Schulhoff et al. [19], the operation the model will perform. This requirement is represented by the **Main Instruction** node. Since a single prompt can contain multiple operating directives, this node is designed to accommodate a list of instructions, not just a single element.

Context

For complex tasks, the core instruction is insufficient in isolation, specific contextual informations become mandatory to rigorously define the scope and constraints of the desired response. This led to the definition of the **Context** node. During the analysis, it emerged that the concept of “context” acts as a generic container for different types of auxiliary information. To understand this, we simply need to consider how many times during the interaction with an LLM we ask it to provide output in a certain format, impose constraints on the response, provide it with data to work with, and so on. To ensure the granularity necessary for any text analysis or modification operations, the **Context** node has been decomposed into **four semantic subnodes**:

- **Format:** Dedicated to directives that define the physical and stylistic structure of the output (e.g., “write a list”, “use JSON format”, “use a formal tone”).
- **Constraints:** Dedicated to portions of text that impose negative limits or boundaries on generation (e.g. “do not use technical jargon”, “only take into account the information provided”).
- **Data:** It contains the data provided by the user that the model must process in performing the task (e.g. a text to be summarized, a code fragment to be analyzed)
- **Role:** Derived from the Role Playing technique, this node isolates instructions relating to the “persona” or professional role that the model must adopt (e.g. “act as a cybersecurity expert”).

Some of the sections identified by Schulhoff et al. [19] and considered in a scattered manner within the data structure are grouped into a single macro-category, **Context**. Figure 3.13 shows how this division occurs with some of the identified components

Act as if you were an expert React programmer. Write the code to create a Navbar.
Make sure it's responsive, and only provide us with the required code as output.

ROLE Act as if you were an expert React programmer. **MAIN INSTRUCTION** Write the code to create a Navbar.
CONTEXT → CONSTRAINTS Make sure it's responsive, and only provide us with the required code as output.

Figure 3.13: Example of base structure

3.4.2 Shot-Based Components

Analysis of the *In-Context Learning* mechanism highlighted the need to manage user-provided examples to guide the model. These examples are semantically outside the main instruction (they are not the task to be performed) nor within the context (they are not constraints or data to be processed). Consequently, the **Examples** node was introduced [19]. It is structured to contain a list of subnodes ($E_1, E_2, \dots, E_n$), allowing for the isolation of individual shots. Figure 3.14 shows how this splitting occurs with shot-based components. This separation is critical; during testing, it allows a single example to be modified, permuted, or removed without altering the logical structure of the rest of the prompt.

You are a SQL expert. Convert the following natural language requests into valid PostgreSQL queries.
Request: Show me all users who signed up in the last 30 days. SQL: SELECT * FROM users WHERE signup_date >= CURRENT_DATE - INTERVAL '30 days';
Request: Find the average order value for each category. SQL: SELECT category, AVG(price) FROM orders GROUP BY category;
Request: List the names of employees who earn more than their managers. SQL:

ROLE You are a SQL expert. **MAIN INSTRUCTION** 1: Convert the following natural language requests into valid PostgreSQL queries.
2: Request: List the names of employees who earn more than their managers. SQL:
EXAMPLES Request: Show me all users who signed up in the last 30 days. SQL: SELECT * FROM users WHERE signup_date >= CURRENT_DATE - INTERVAL '30 days';
Request: Find the average order value for each category. SQL: SELECT category, AVG(price) FROM orders GROUP BY category;

Figure 3.14: Example of a structure with shot-based components

3.4.3 Reasoning-Based Components

The introduction of advanced techniques that induce explicit reasoning within the model, such as Chain-of-Thought and Tree-of-Thoughts, has led to the inclusion of elements that cannot be considered either simple instructions or context. To classify these components, the **Reasoning** node was defined, further subdivided to handle the different paradigms:

- **Influence:** Designed for **Zero-Shot CoT** and **Tree-of-Thoughts**, this node captures so-called *trigger phrases* that induce the model to reason, such as “*Let’s think step by step*” for the CoT (Chain-of-Thought) method, or the directives present in the ToT (Tree-of-Thoughts) method that instruct the model to generate different reasoning possibilities (“Generate multiple diagnoses”).
- **Reasoning Examples:** Designed for **Few-Shot CoT**, this node will contain all task execution examples that include reasoning steps. This node will also allow the insertion of several subnodes ($R_1, R_2, \dots$), one for each example, for the same reasons described in the section on the **Examples** node.
- **Paths:** Used in the *Tree-of-Thoughts*, this node contains the specific branches or initial options that the model should explore or evaluate ($P_1, P_2, \dots$).

Figure 3.15 shows the split with a reasoning component

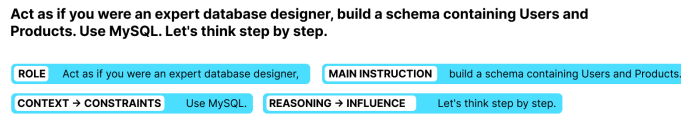


Figure 3.15: Example of structure with “think” components

3.4.4 Robustness to Advanced Prompting Techniques

The analysis conducted on advanced prompting techniques, described in the State of the Art, confirmed the robustness of the proposed framework. It is worth noting that the adoption of complex paradigms does not require the introduction of additional node types within the PST.

Iterative and Multi-Turn Prompting.

Techniques such as **Self-Refine** or **Multi-Turn** approaches handle conversations consisting of question-and-answer sequences. Although these methods involve managing a conversational memory and iteratively updating the prompt, they do not structurally introduce new semantic categories.

Information from previous turns, as well as refinement feedback, are incorporated into the current prompt as supporting information. Consequently, within the PST, these elements are naturally classified as *Data* within the *Context* node: the model receives the conversation history as data on which to base the next response.

Representation-Based Prompting.

Methodologies such as **Code Prompting** or *Program-of-Thoughts* aim to improve model performance by requiring the output to be represented in executable code rather than natural language. Again, the structure of the PST remains unchanged. The request to generate code acts as a constraint on the representation, mapping seamlessly into existing **Main Instruction** nodes (e.g., “Write a Python function...”) or **Format** (e.g., “Answer using a block of code”). The nature of the task changes, but the syntactic structure of the prompt to communicate it falls within the already defined categories. The same reasoning also applies to other techniques such as **Chain-of-Table**, which use the same paradigm but with different representations (in this case, a tabular representation).

3.5 Formal Specification

3.5.1 Hierarchical Structure

Unlike an **Abstract Syntax Tree** (AST) for programming languages, where the hierarchical structure represents the syntactic succession of the source code, which is possible due to their constraints, the **Prompt Syntax Tree** (PST) models natural language, which lacks such rigid sequential constraints. In a natural language prompt, semantically related instructions may be fragmented and dispersed across the text. For instance, the definition of the output format might be split between the beginning and the end of the prompt, separated by unrelated sections. To address this problem, the PST adopts a **Logical Aggregation** strategy. Rather than creating a new structural node each time a semantic category reappears in the text sequence (which would result in a fragmented and redundant tree), as is done in an AST, the PST defines unique category nodes (e.g., a single *Format* node) that serve as containers. Conse-

quently, each leaf node in the schema is formally defined not as a single string, but as a **list of segments**. This design choice ensures that automated tools can access all information related to a specific category (e.g., formatting constraints) by traversing a single path, regardless of how sparse that information was in the original input.

Let's take this prompt for example Figure 3.16:

Act as if you were an expert in, generate the output as a list, consider only the information I provided, use bold for important information, and stick to the context described in the information. Given this data "...", how do I perform this operation?

Figure 3.16: Example prompt used to illustrate logical aggregation.

The two hierarchical representations of the text (3.16) are shown in the Figure 3.17.

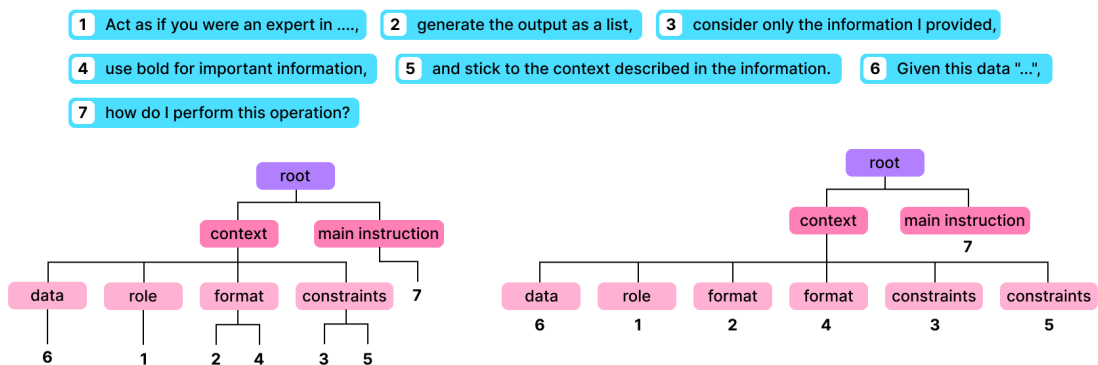


Figure 3.17: Examples of Structure

It is easy to notice how the first representation is much more compact and less dispersive than the second one 3.17.

Based on this principle, the complete schema, shown in the figure 3.18, is defined as follows:

1. **Root:** As root we have the **Prompt** node, which represents the entire text block.
2. **Level 1 Nodes:** Main Instruction, Context, Examples, Reasoning.
3. **Level 2 (Context):** Format, Constraints, Data, Role.

4. **Level 2 (Examples):** List of individual Example nodes (E_n).
5. **Level 2 (Reasoning):** Influence, Reasoning Examples, Paths.

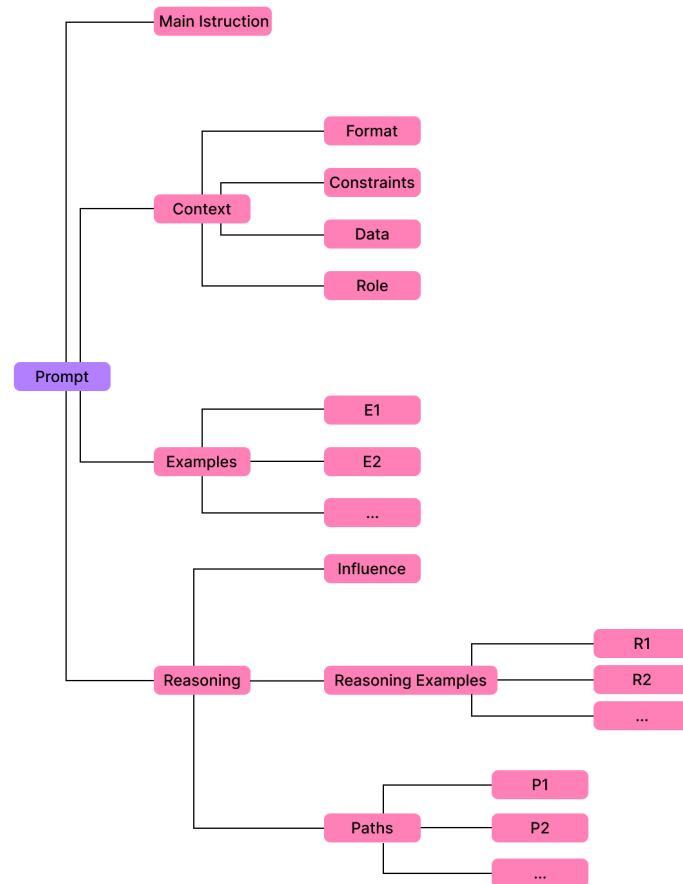


Figure 3.18: Prompt Syntax Tree Structure

3.5.2 Leaf Nodes and Sequence Preservation

While logical aggregation simplifies analysis, it risks losing a critical piece of information, the original linear sequence. Preserving the exact order of instructions is vital for maintaining the logical flow, where any modification must remain contextually coherent with the surrounding text. To reconcile the logical tree structure with the linear nature of the input, the PST employs a specific definition for its leaf nodes. The leaves do not store raw strings directly, instead, they store tuples containing positional data:

$$Leaf = \{\text{position} : \mathbb{N}, \text{phrase} : \text{String}\}$$

Where *phrase* contains the textual content and *position* represents the index of that phrase in the original sequence. This formalization make the PST a **lossless representation**: the original prompt can always be perfectly reconstructed by collecting all leaves and sorting them by their position index. An example of this hierarchical organization with the addition of positional data is shown in Figure 3.19

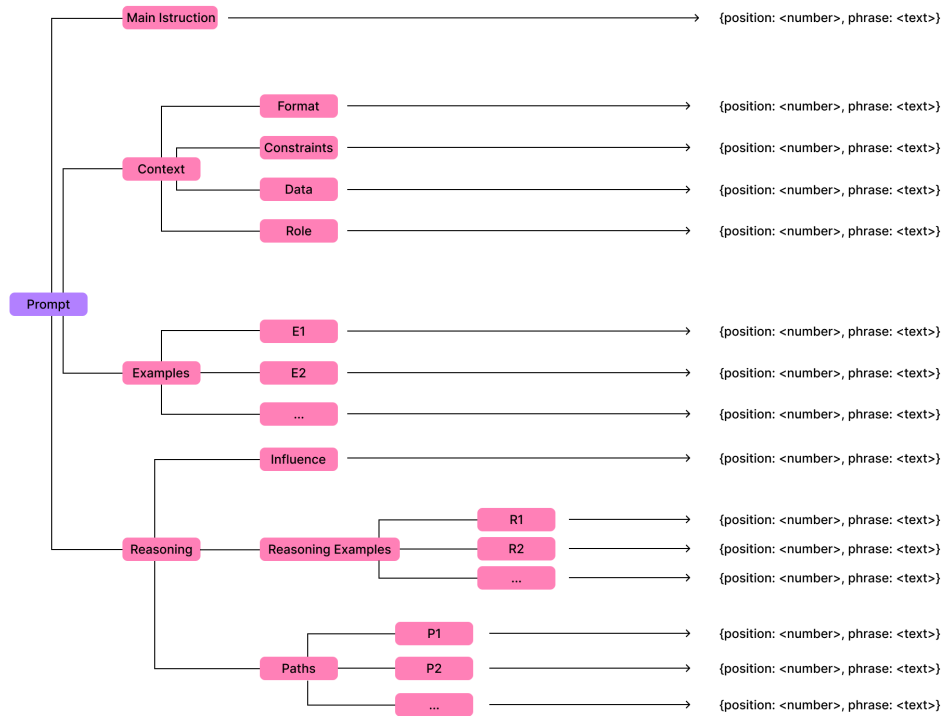


Figure 3.19: Prompt Syntax Tree leaf definition

3.5.3 Construction Strategy: Inference and Alignment

The generation of this structure involves a hybrid approach aimed at optimizing computational efficiency. Training a sequence-to-sequence model to predict exact numerical indices (the *position* field) would introduce unnecessary complexity and increase the computational load, distracting the model from its primary task of semantic segmentation. Therefore, the construction of the PST is divided into two phases:

1. **Semantic Generation:** The neural model is responsible solely for generating the structure and the textual content of the leaves (the *phrase*).
2. **Algorithmic Alignment:** A deterministic post-processing algorithm maps the generated phrases back to the original input string to calculate and inject the correct *position* values.

This strategy ensures high structural accuracy while maintaining the ability to pinpoint the exact location of every segment within the original prompt.

3.6 Data Representation

To make the **Prompt Syntax Tree** computationally usable for machine learning tasks, the abstract hierarchical structure defined in the previous sections must be serialized into a machine-readable format. This serialization serves a dual purpose: it acts as the standardized output target for the sequence-to-sequence model developed in this thesis, and it defines the structure for the labeled dataset used during training. The construction of the dataset necessary for this task involved a rigorous process of manual annotation. Starting from existing open-source corpora, prompts were analyzed and labeled to map their natural language components into the PST definitions. The result is a collection of pairs mapping raw unstructured inputs to their corresponding structured representations.

3.6.1 JSON Schema Definition

The serialization format chosen for the PST is **JSON**. This choice allows for a rigid yet flexible representation of the logical aggregation strategy discussed in Section 3.4.1. Unlike a recursive parse tree, the PST JSON schema is designed as a **flat hierarchy of semantic categories**. The root object contains keys corresponding strictly to the Level 1 nodes defined in the formal specification: `main_instruction`, `context`, `examples`, and `reasoning`, within which, where present, are contained the Level 2 nodes described in the same section: `format`, `constraints`, `data` and `role` for **Context** and `influence`, `reasoning_examples`, `paths` for **Reasoning**. Inside these categories, the schema enforces the structure of *leaf nodes*. Every textual element is represented not as a string, but as a positional object tuple:

`{"position" : Integer, "phrase" : String}`

Figure 3.20 illustrates the complete structure of a generic PST instance.

```
{
  "main_instruction": [
    {
      "position": 0,
      "phrase": "Summarize the following text"
    }
  ],
  "context": {
    "format": [
      {
        "position": 2,
        "phrase": "Use a bulleted list"
      }
    ],
    "constraints": [
      {
        "position": 3,
        "phrase": "Do not exceed 100 words"
      }
    ],
    "data": [
      {
        "position": 1,
        "phrase": "<input_text_content>"
      }
    ],
    "role": []
  },
  "examples":
    // Array of examples, where each example
    // is a separate object
    [
      {
        "position": 10,
        "phrase": "Input: ... Output: ..."
      },
      {
        "position": 11,
        "phrase": "Input: ... Output: ..."
      }
    ],
  "reasoning": {
    "influence": [
      {
        "position": 4,
        "phrase": "Let's think step by step"
      }
    ],
    "reasoning_examples": [],
    "paths": []
  }
}
```

Figure 3.20: Prompt Syntax Tree JSON Object

This representation ensures that all semantic components derived in Section 3.3, including complex reasoning structures like ToT paths, are captured in a format that preserves both the semantic classification and the original linear order of the prompt.

3.7 Summary

In this chapter, we have introduced the *Prompt Syntax Tree* (PST), a novel formal representation designed to overcome the limitations of unstructured text prompts. By drawing an analogy with the Abstract Syntax Tree (AST) used in compiler theory, we defined a hierarchical structure that decomposes a prompt into its constituent functional components: *Main Instruction*, *Context*, *Examples*, and *Reasoning*. Of these components, the Context and Reasoning nodes are further broken down into subnodes to increase the granularity of the structure. These are divided into **Format**, **Constraints**, **Data**, and **Role** for the context node, and **Influence**, **Reasoning Examples**, and **Path** for the Reasoning node, respectively. The derivation of these components was strictly guided by the taxonomy of prompting techniques (Zero-shot, Few-shot, CoT, ToT) analyzed in Chapter 2. Furthermore, we addressed the specific challenge of *natural language fragmentation* by adopting a **logical aggregation** strategy and a **position-aware leaf** definition. This ensures that the PST is a lossless representation, allowing for the reconstruction of the original sequence. Finally, we formalized the data representation into a JSON schema. With this theoretical framework established, the following chapter will detail the implementation of the dataset creation process and the training of a sequence-to-sequence model capable of automatically generating PSTs from raw input prompts.

Implementation and Model Training

4.1 Introduction

In the previous chapter, we formalized the Prompt Syntax Tree (PST) as a theoretical framework for representing the semantic structure of LLM prompts. However, a theoretical representation alone is insufficient to address the "Promptware Crisis" identified in the literature. To operationalize the PST for practical software engineering tasks we require a computational mechanism capable of translating raw, unstructured text into this rigorous hierarchical format.

This chapter details the engineering of that mechanism: an automated extraction tool powered by a Fine-Tuned LLM. We transition here from the abstract definition of the schema to the practical implementation of a supervised learning pipeline. The following sections describe the end-to-end workflow, encompassing the acquisition of domain-specific data from DevGPT [17], the curation of a high-quality ground truth via manual annotation and synthetic augmentation, and finally, the supervised fine-tuning (SFT) of the Llama-3 architecture to perform the translation task.

4.2 Overview of the Training Pipeline

This chapter details the engineering process developed to train the sequence-to-sequence model responsible for generating Prompt Syntax Trees (PST). The workflow follows a standard machine learning pipeline, consisting of four distinct stages: Data Acquisition, Data Annotation and Augmentation, Model Selection, and Supervised Fine-Tuning (SFT).

4.3 Dataset Acquisition and Preparation

4.3.1 Source Selection

To ensure the model is specialized in the software engineering domain, the training data was sourced from **DevGPT**, a dataset created in the Della Porta et al. [17] research. This dataset contains user-ChatGPT conversations mined from software development contexts, including GitHub issues, pull request, discussions, commits, code files and Hacker News threads. The original dataset comprises 6 JSON files representing all the different sources, totaling 5,494 conversations and approximately 29,788 prompts. The authors of the DevGPT paper performed a preliminary manual review of the retrieved data, where they found the presence of duplicate conversations and included generic conversations that fell outside the scope of software development. Consequently, they applied a data preparation process, where they removed duplicate conversations and applied a topic classification to filter out non-technical conversations, ensuring high relevance to software tasks.

4.3.2 Single-Turn Extraction Strategy

The DevGPT dataset consists of multi-turn conversations. However, the PST framework defined in Chapter 3 operates on the *atomic unit* of a single prompt. Training on intermediate conversation turns could introduce referential ambiguities (e.g., “rewrite that function”) where the context is missing from the prompt itself. To address this, a **Single-Turn Extraction Strategy** was applied. Only the **initial user message** of each conversation was extracted. This decision ensures that every data

point in our training set is a self-contained, “finite” prompt containing all necessary context, maximizing the model’s ability to learn structural segmentation without external dependencies. This process resulted in approximately 1,000 high-quality prompts per source file.

4.3.3 Data Cleaning

Following extraction, a manual analysis revealed the presence of multi-lingual data (Chinese, Italian, French, etc.). To maintain consistency in the linguistic patterns learned by the model, a cleaning operation was performed to remove non-English prompts, resulting in a monolingual English dataset.

4.4 Data Annotation and Augmentation

4.4.1 Labeling Process and Annotation Guidelines

The filtered prompts were exported to a spreadsheet format for manual annotation. To ensure semantic consistency across the dataset, the labeling was performed manually by the author, adhering to a strict set of operational guidelines derived from the PST taxonomy defined in Chapter 3.

Each prompt was segmented into the leaf nodes of the syntax tree based on the following annotation criteria:

- **Main Instruction:** identifies the core imperative sentences. Any text specifically requesting the LLM to perform an action or execute a task was mapped here.
- **Context - Data:** captures the passive information provided for processing. All text containing the raw content (e.g., code snippets, text to summarize) that the model acts upon was classified as data.
- **Context - Role:** includes all sentences that explicitly assign a persona, profession, or simulation scenario to the model (e.g., “Act as a Senior Engineer”).
- **Context - Constraints:** defined as restrictive or proscriptive boundaries. This category includes all sentences imposing limits on the output space, such as

negative constraints (“do not use...”), exclusions, or hard limits that the model must respect.

- **Context - Format:** defined as prescriptive or positive instructions regarding presentation. Unlike constraints, these sentences instruct the model on how to present the generated output, including directives on tone (e.g., “formal style”) or specific structural constructs (e.g., “markdown table”, “JSON format”).
- **Examples:** maps standard few-shot learning components. This includes text providing Input-Output pairs from which the model infers the task pattern without explicit reasoning steps.
- **Reasoning - Influence:** captures “trigger phrases” designed to activate specific reasoning modes (e.g., “Let’s think step by step”).
- **Reasoning - Reasoning Examples:** distinguishes Chain-of-Thought (CoT) examples from standard examples. This category includes Input-Reasoning-Output triples, where the example explicitly demonstrates the intermediate logical steps required to reach the solution.
- **Reasoning - Paths:** specific to the Tree-of-Thoughts (ToT) paradigm. This captures text specifying distinct perspectives or alternative logical branches the model must explore.

Given the inherent flexibility of natural language, certain instructions presented semantic ambiguity (e.g., overlapping definitions of format vs. constraint). To resolve these instances, a comparative analysis strategy was employed: ambiguous segments were cross-referenced against previously labeled prompts within the dataset to maintain internal congruency and strict intra-annotator consistency.

Finally, during this phase, a length filter was applied. Prompts exceeding the context window limit of the target 8B model were discarded to ensure training efficiency and stability.

4.4.2 Synthetic Data Generation

During the initial labeling phase, a significant class imbalance was detected within the dataset. As anticipated by the empirical findings of Della Porta et al. [17], developers exhibit a strong preference for simple “Zero-Shot” prompts, resulting in a dataset heavily skewed towards direct instructions with a scarcity of complex reasoning structures.

This distribution poses a risk of mode collapse, where the model might fail to learn the PST structures associated with advanced prompting paradigms simply due to a lack of exposure. To mitigate this and ensure the model generalizes across the full PST taxonomy, a Synthetic Data Augmentation strategy was implemented.

The process was executed as follows:

- **Seed Creation:** we manually crafted a seed dataset of 30 high-quality prompts specifically designed to trigger complex reasoning behaviors. These seeds covered three specific categories: Zero-Shot Chain-of-Thought, Few-Shot Chain-of-Thought, and Tree-of-Thoughts.
- **Generation:** the generation process was executed via the **NVIDIA NIM (NVIDIA Inference Microservices) API**. Utilizing the ChatNVIDIA endpoints within a Google Colab environment, we leveraged the Llama-3-70B-Instruct model to expand the dataset. Using the manual seeds as stylistic and structural templates, we generated a total of 150 new unique prompt examples, specifically distributed as 50 Zero-Shot CoT, 50 Few-Shot CoT, and 50 Tree-of-Thoughts prompts.
- **Validation:** the synthetic prompts were manually reviewed to ensure they logically adhered to the target paradigms before being labeled and injected into the training set.

Table 4.1 shows examples of the manually crafted seeds used to guide the generation.

Seed Prompt Example		
Paradigm	Seed Prompt	
	Example 1	Example 2
Zero-Shot CoT	Calculate the Time and Space complexity of the following recursive Java function for traversing a binary tree. Instruction: Don't just give the Big-O notation. Step through the recursion stack depth for a balanced vs. skewed tree. Analyze the auxiliary...	think of all the similar data types for this purpose and general message structures and create a ros 2 message type and its fields. this message will be used by specific nodes to report their system resource usages
Few-Shot CoT	Identify the correct HTTP code. User: User tries to access /admin but is not logged in. Model: Thought: Authentication is missing. Code: 401 Unauthorized....	Write unit tests for the provided function. User: Function: sum(a, b) return a + b; Model: Thought: I need to test the happy path (positive integers), negative numbers, and zero. Test: JavaScript... ..
Tree-of-Thoughts	You are a CTO designing a social media feed system. We need to choose a database technology to store user activity logs (likes, clicks, views). Input: High write throughput (100k/sec), flexible schema, eventual consistency is acceptable. Process:...	I am starting a large-scale React E-commerce application. I need to decide on a State Management strategy. Generate Options: React Context + useReducer only. Redux Toolkit (RTK)....

Table 4.1: Examples of Prompts used as Seed for the generation of synthetic data

4.4.3 JSON Serialization

The final step of data preparation involved converting the labeled columnar data into the target JSON structure defined in *Chapter 3*. The labeling process produced an intermediate Excel artifact where each row represented a prompt and each column corresponded to a PST leaf node (e.g., `main_instruction`, `constraints`, `data`). To transform this flat representation into the hierarchical PST format, a custom Python serialization script was developed. The script operates by iterating through the Excel rows and constructing a JSON object composed of two primary keys:

- `prompt`: the raw, unstructured input string (reconstructed by concatenating the parts).
- `json`: the structured PST object.

Handling Logical Aggregation (<sep>)

A specific challenge addressed during this phase was the fragmentation of semantic categories. As defined in the **Logical Aggregation** strategy (Section 3.4.1), a single semantic component (e.g., `constraints`) might appear in multiple disjoint parts of the source text. During labeling, these disjoint parts were inserted into the same Excel cell, separated by a special token: `<sep>`. The serialization script includes a parsing logic that checks for the presence of `<sep>` in every cell. If detected, the string is split, and the resulting segments are appended as distinct objects within the corresponding JSON array field (e.g., creating multiple objects inside `context.constraints`). This ensures that the JSON structure faithfully represents the fragmentation of the original natural language.

4.4.4 Dataset Evaluation and Quality Assurance

Given that the dataset was constructed through a semi-automated process involving manual labeling and synthetic generation, a rigorous quality assurance step was necessary to rule out human error or data corruption. We implemented an automated **Integrity Check** using a custom validation script. The script performs a bidirectional comparison for every entry in the final JSON dataset:

- **Source-to-Target Verification:** it compares the “raw prompt” string against the aggregated content of the generated PST leaf nodes.
- **Lossless Check:** by stripping all whitespace and special formatting characters (carriage returns, tabs), the script verifies that the length and character composition of the source string match the reconstructed string exactly.

This verification ensures that no text was accidentally omitted during the copy-paste operations of the labeling phase and that the `<sep>` splitting logic did not result in data loss. Entries that failed this check were flagged for manual correction.

4.5 Model Selection

For the backbone of our architecture, we selected **Llama-3.1-8B-Instruct**. This choice was driven by four critical factors that align with the project requirements.

4.5.1 Semantic Disambiguation Capabilities

A key challenge in PST segmentation is distinguishing between semantically similar but functionally distinct instructions. For example:

- *Phrase A:* “The output must be in JSON.” ($\rightarrow$ Constraint)
- *Phrase B:* “Format the response as a JSON object.” ($\rightarrow$ Format)

Research by Wei et al. [33] on *emergent abilities* demonstrates that the capacity for such complex reasoning and symbol manipulation is not linear; rather, it typically emerges as a function of model scale. Their findings indicate that smaller models often perform near-randomly on reasoning tasks, while performance improves significantly once a certain threshold of parameter count and training compute is surpassed.

Consequently, to maximize the model’s ability to discern these subtle semantic nuances, it was necessary to select the largest architecture computationally viable. While earlier literature identified these capabilities primarily in models exceeding 60B parameters, recent advancements in dense model training (such as Llama 3) have pushed these capabilities down to smaller sizes. Therefore, we selected the 8B

parameter variant, as it represents the upper bound of what could be fine-tuned within our experimental environment (see Section 4.4.3), balancing the theoretical need for scale with practical hardware limitations.

4.5.2 Complex Instruction Following

The task requires the model to simultaneously adhere to multiple strict rules: splitting sentences, mapping to specific JSON fields, handling null values, and avoiding text duplication. We selected an Instruct-tuned variant (RLHF) rather than a base model.

As demonstrated by Ouyang et al. [34] in their analysis of InstructGPT, base large language models are trained merely to predict the next token on a vast corpus of text, which often results in “misaligned” behaviors, such as making up facts or failing to follow explicit directives. In contrast, models fine-tuned with Reinforcement Learning from Human Feedback (RLHF) show a significantly improved ability to align with user intent and adhere to explicit constraints. This capability is critical for our task, where the model must act as a strict functional parser rather than a creative text generator.

4.5.3 Computational Feasibility

The training was conducted in a cloud environment (Google Colab) with hardware limitations. The 8B parameter architecture represents the optimal trade-off, maximizing reasoning capabilities while remaining trainable within the available VRAM constraints using quantization techniques.

4.6 Training Methodology

4.6.1 Chat Format and System Prompt

To fully leverage the capabilities of the Instruct model, the training data was formatted using a conversational template. This transforms the instruction from a passive suggestion into an active constraint. Crucially, we engineered a specific

System Prompt, showed in Listing 4.1, included in every training example. This prompt acts as a **learning weight**, authoritatively defining the rules of the PST segmentation.

Listing 4.1: System prompt used for the model fine-tuning.

You are an expert Prompt Engineering Analyst. Your task is to analyze the user's input prompt and perform a structural segmentation based on the Prompt Structure Tree (PST) framework.

You must output a valid JSON object containing the exact text segments from the input, categorized into the schema defined below.

Schema Definitions

1. ****main_instruction****: The core task or action the model is asked to perform.
2. ****context****:
 - ****format****: ****Positive instructions**** defining the target structure, style, or specific layout. Describes what the output *should be* (e.g., "Format as JSON", "Use a list").
 - ****constraints****: ****Restrictive instructions**** that limit the generation. This includes negative constraints ("do not use..."), exclusions, hard limits ("max length"), or strict requirements marked by words like "only" or "strictly" (e.g., "Output ONLY in JSON", "No markdown").
 - ****data****: The raw input content to be processed.
 - ****role****: Persona definitions.

3. **examples**: Input-output pairs used for few-shot learning.
4. **reasoning**: Components related to Chain-of-Thought (CoT), divided into:
 - **influence**: Triggers for CoT (e.g., "Think step by step") and ToT, instructions to generate multiple solution paths.
 - **reasoning_examples**: Few-shot examples specifically showing reasoning steps.
 - **paths**: Specific branches or alternative options provided for reasoning.

Rules

1. **Exact Extraction**: Extract text exactly as it appears in the prompt. Do not paraphrase.
2. **Mutual Exclusivity**: Each phrase or section of the prompt must be inserted into exactly one field. Do not duplicate text across multiple fields.
3. **Sub-Sentence Splitting**: You must break sentences apart if they contain mixed instructions. For example, if a sentence defines a Role and then gives a Constraint, split the text at the transition point.
4. **Preserve Continuity**: Ensure that if the segments were concatenated back together, they would reconstruct the original text (including punctuation and conjunctions).
5. **Null Handling**: If a section is missing from the input, return ``null`` (for single fields) or ``[]`` (for lists).
6. **JSON Validity**: Ensure the output is strictly valid JSON without Markdown formatting (no ````json` blocks).

Input Handling

Analyze only the text provided within the ``<input_prompt>`` tags .

To ensure the model adopts the specific analytical behaviors required for the PST framework, the System Prompt was engineered to function as a set of rules rather than a mere suggestion. It explicitly addresses several critical challenges in the segmentation task:

- **Semantic Demarcation:** it provides rigorous definitions to help the model disambiguate semantically adjacent categories, specifically distinguishing between Format (defined as 'positive' instructions for layout) and Constraints (defined as 'restrictive' instructions for boundaries).
- **Operational Integrity:** it enforces the principle of **Exact Extraction**, requiring the model to preserve the original text verbatim. This is coupled with a strict **Mutual Exclusivity** rule to prevent content duplication across fields.
- **Granular Segmentation:** the prompt explicitly instructs the model on handling complex syntactic structures, authorizing the splitting of single sentences into multiple fields when they contain mixed instructions, while simultaneously mandating that the logical continuity of the original prompt be preserved.
- **Syntactic Validity:** finally, it standardizes the output format by defining rules for handling null values and ensuring the generation of strict, valid JSON objects.

By embedding these constraints directly into the system message, we reduce the search space for the model during gradient descent, thereby significantly improving convergence times.

4.6.2 Hyperparameters and Configuration

The model was fine-tuned using the **Unsloth** framework, which optimizes back-propagation for efficiency. We utilized a train/evaluation split of 90/10, with 109

examples reserved for validation. The training configuration was set as follows:

- **Batch Size:** 2 (per device) with Gradient Accumulation of 4.
- **Learning Rate:** $2e-4$.
- **Optimizer:** AdamW 8-bit.
- **Max Steps:** 300.
- **Precision:** FP16 (with BF16 fallback).

4.7 Preliminary Results

The training process demonstrated stability and rapid convergence. The model achieved a final **Training Loss of 0.2** and a **Validation Loss of 0.6**. These metrics indicate that the model successfully learned the syntactic structure of the PST without significant overfitting, validating the effectiveness of the single-turn extraction and synthetic augmentation strategies.

Empirical Evaluation

5.1 Objectives

The goal of this empirical evaluation is to validate the two fundamental contributions of this thesis: the **Prompt Syntax Tree** (PST) as a formal representation and the **fine-tuned language model** as an automated extraction tool. While the theoretical design of the PST was grounded in a comprehensive taxonomy of prompting techniques, its practical adequacy cannot be assumed a priori. Similarly, the efficacy of the developed model must be rigorously quantified, not merely in terms of text generation, but as a reliable parser capable of supporting downstream engineering tasks.

To guide this evaluation, we formulated three research questions that investigate distinct dimensions of the proposed solution, ranging from **structural coverage** to **semantic precision**.

***RQ₁** - To what extent is the Prompt Syntax Tree (PST) capable of representing diverse prompting paradigms without semantic loss?*

This question aims to verify whether the hierarchical structure defined in *Chapter 3* is sufficiently expressive to capture the nuances of real-world prompts, including

complex paradigms like Chain-of-Thought, without discarding critical information or forcing data into incorrect categories. A high degree of coverage is a prerequisite for the validity of any subsequent analysis.

RQ₂ - How effective is the fine-tuned model in automatically decomposing unstructured prompts into valid PST representations?

This question focuses on the automation capability. It seeks to quantify the model’s reliability in generating syntactically correct JSON objects and its accuracy in segmenting raw text into the appropriate leaf nodes.

RQ₃ - How accurately does the model distinguish between semantically similar but functionally distinct components?

This question investigates the model’s depth of understanding. In prompt engineering, the boundary between a Constraint (e.g., “Do not use jargon”) and a Format instruction (e.g., “Write in a formal tone”) can be subtle. By analyzing the confusion between predicted labels, we aim to determine if the model has learned the underlying logic of the PST taxonomy or if it struggles to disambiguate these semantically adjacent concepts.

5.2 Context of the Study

To answer these research questions, we designed an experimental protocol that aligns with the **validation strategies** in software engineering for generative AI. Specifically, we adopt a **reference-based evaluation methodology**, treating the prompt decomposition task as a unit test where model outputs are rigorously compared against a ‘golden’ ground-truth dataset.

Dataset and Partitioning

The evaluation was conducted using the dataset prepared in *Chapter 4*, which was derived from the DevGPT corpus [17]. As described in Section 4.5.2, the full dataset was split into training and evaluation sets. For this empirical study, we utilized the held-out test set of *109 prompts* (10% of the dataset). These prompts

were explicitly excluded from the training phase to ensure that all metrics reflect the model’s generalization capability on unseen data rather than memorization. The test set was curated to include a balanced distribution of prompting paradigms, ranging from simple **Zero-Shot** requests to complex **Few-Shot** and **Reasoning-heavy inputs**.

Baseline and Ground Truth

Since the automatic decomposition of prompts into PST structures is a novel task, there are no existing baselines (e.g., prior models or tools) for direct comparison. Consequently, the performance of our fine-tuned model is evaluated against the **Human Annotated Ground Truth**. These annotations serve as the gold standard, created during the manual labeling process described in *Section 4.3.1*. The comparison between the Model Generated Output and the Human Annotation provides the basis for all calculated metrics.

Hardware and Environment

The inference and evaluation pipeline were executed in the same cloud environment used for training (Google Colab), leveraging the optimized inference capabilities of the Unsloth framework. This setup ensures that the reported performance metrics are consistent with the computational constraints discussed in *Section 4.4.3*, validating the feasibility of the tool for resource-constrained scenarios.

5.3 Data Collection Procedure

To gather the data necessary to answer the research questions, we executed a standardized inference pipeline on the test set. The data collection process was performed as follows:

- **Inference Execution:** the fine-tuned Llama-3 model was deployed using the Unsloth inference engine. We fed the model the 109 held-out test prompts (unseen during training) using the system prompt defined in *Section 4.5.1*.
- **Artifact Generation:** for each input prompt x_i , the model generated a corresponding JSON output y_i .

- **Extraction and Alignment:** the generated JSONs were parsed to extract the predicted semantic segments (e.g., text classified as ‘Constraints’ or ‘Data’). These predictions were then aligned token-by-token with the Ground Truth manual annotations to identify matches, mismatches, and hallucinations.

5.4 Evaluation Metrics

To rigorously assess the performance of the model, we adopted a multi-dimensional evaluation strategy. Since the task involves both structured generation (producing valid JSON) and information extraction (segmenting text into specific categories), standard text generation metrics like BLEU or ROUGE are insufficient. Instead, we defined a set of metrics to answer the three research questions.

5.4.1 Analysis for RQ_1

To answer RQ_1 , we analyze the model’s ability to produce valid and complete PSTs. To verify this we used three specific metrics.

Structural Validity

The primary requirement for the automated tool is the generation of a machine-readable artifact. The JSON Validity Rate (V_{rate}) measures the model’s ability to produce syntactically correct JSON objects that adhere to the PST schema defined in Chapter 3.

$$V_{rate} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(isValid(y_i))$$

Where:

- N is the total number of prompts in the test set.
- y_i is the output generated by the model for the i -th prompt.
- $\mathbb{1}(isValid(y_i))$ is an indicator function that returns 1 if y_i is a valid JSON object containing the required root keys (main_instruction, context, etc.), and 0 otherwise.

Content Reliability

Since the goal is to decompose a prompt without altering its content, we introduce two metrics to measure the model’s adherence to the source text.

1. **Hallucination Rate (HR):** This metric quantifies the generation of *phantom* content, words that appear in the output JSON but were not present in the input prompt. A reliable extractor should have an HR near zero.

$$HR = \frac{|T_{gen} \setminus T_{src}|}{|T_{gen}|}$$

Where T_{gen} is the set of tokens in the generated JSON values, and T_{src} is the set of tokens in the source prompt.

2. **Coverage Score (CS):** This metric measures how much of the original prompt was successfully captured and assigned to a node in the PST tree. It serves as a proxy for the “lossless” nature of the representation.

$$CS = \frac{|T_{src} \cap T_{gen}|}{|T_{src}|}$$

A Coverage Score of 1.0 (100%) implies that every word in the user’s prompt was successfully mapped to a specific component in the syntax tree, validating the expressiveness of the PST schema.

5.4.2 Analysis for RQ_2

To answer RQ_2 , which concerns the accurate separation of semantic blocks, we treat the evaluation as a classification task.

Semantic Segmentation Performance: Field-Level F1

To verify the accuracy of the segmentation, we treat the problem as a token classification task. For each semantic field f in the PST (e.g., Main Instruction, Context.Data), we compute the **Token-Level F1 Score**.

Let S_f be the set of tokens (words) assigned to field f in the ground truth (human annotation), and $\hat{S}_f$ be the set of tokens assigned to field f by the model. We define Precision (P_f) and Recall (R_f) as follows:

$$P_f = \frac{|S_f \cap \hat{S}_f|}{|\hat{S}_f|} \quad , \quad R_f = \frac{|S_f \cap \hat{S}_f|}{|S_f|}$$

- **Precision measures reliability:** of the words the model claimed were “Constraints”, how many actually were?
- **Recall measures completeness:** of the words that were actually “Constraints”, how many did the model find?

The **F1 Score** for field f is the harmonic mean:

$$F1_f = 2 \cdot \frac{P_f \cdot R_f}{P_f + R_f}$$

This metric allows us to identify specific weaknesses in the model (e.g., if the model performs well on extracting Instructions but poorly on separating Role).

5.4.3 Analysis for RQ_3

To answer RQ_3 , regarding the subtle distinction between functionally similar components (e.g., Format vs. Constraints), we used a **Token-Level Confusion Matrix**.

Component Disambiguation: Confusion Matrix

One of the most critical challenges in prompt engineering is distinguishing between semantically similar components, such as a Format directive (e.g., “Write a list”) versus a Constraint (e.g., “Do not use paragraphs”).

To analyze these errors, we construct a **Token-Level Confusion Matrix**. By aligning the generated text back to the original prompt, we calculate the number of tokens that belong to class i in the ground truth but were predicted as class j by the model.

Rows represent the **True Labels** (Human Annotation), and columns represent the

Predicted Labels (Model Output). Off-diagonal elements represent specific types of *semantic confusion*, allowing us to diagnose if the model systematically misclassifies specific PST components.

CHAPTER 6

Analysis of the Results

This chapter presents the findings of our empirical evaluation, organized to directly answer the three research questions formulated in Section 5.1. The results are derived from the performance of the fine-tuned Llama-3.1-8B-Instruct model on the held-out test set of 109 prompts. We first address the structural validity and expressiveness of the PST (RQ_1), then analyze the segmentation effectiveness (RQ_2), and finally investigate the semantic disambiguation capabilities (RQ_3).

6.1 Results for RQ_1

To answer RQ_1 , we investigated whether the PST framework, combined with our fine-tuned model, provides a reliable and lossless representation of diverse prompting paradigms. We analyzed the model’s ability to generate syntactically valid JSON objects and whether its output preserved the original text without hallucinations or omissions.

6.1.1 Structural Reliability and JSON Validity

Our analysis reveals that the model achieved a Structural Validity (V_{rate}) of 100%. Across all 109 test cases, the model successfully generated syntactically correct JSON objects that adhered to the PST schema defined in Chapter 3. This indicates that the System Prompt engineering (4.5.1) and the fine-tuning process were highly effective in constraining the model’s generation.

6.1.2 Content Reliability and Coverage

Regarding the “lossless” nature of the representation, the results, shown in the table 6.1, demonstrate a near-perfect adherence to the source text.

- **Coverage Score (CS):** the model achieved a coverage score of 99.91%. This implies that virtually every token from the original unstructured prompts was successfully captured and assigned to a specific node within the PST structure.
- **Hallucination Rate (HR):** simultaneously, the model exhibited a negligible Hallucination Rate of 0.18%. This confirms that the model acts as a faithful extractor, resisting the tendency to generate phantom content or paraphrase the user’s intent.

Metric	Result	Interpretation
Structural Validity	100%	The model generated syntactically perfect JSON for all 109 test cases.
Coverage Score (CS)	99.91%	Virtually all source tokens were successfully mapped to the tree.
Hallucination Rate (HR)	0.18%	The generation of non-existent text was negligible.

Table 6.1: Structural validity and faithfulness metrics for RQ₁.

Summary for RQ_1

The empirical evaluation confirms that the Prompt Syntax Tree (PST) combined with the fine-tuned model provides a robust and lossless representation of unstructured prompts. The model achieved a **JSON Validity Rate of 100%**, eliminating structural errors, while the **Coverage Score of 99.91%** and a negligible **Hallucination Rate of 0.18%** demonstrate that the framework captures the user’s intent verbatim without omission or fabrication.

6.2 Results for RQ_2

RQ_2 evaluates the effectiveness of the model in automatically decomposing prompts into their constituent semantic blocks. Table 6.2 reports the Token-Level F1 scores for the high-level semantic categories.

PST Component	F1 Score	Performance Tier
<code>prompt.examples</code>	1.0000	Perfect
<code>prompt.context.data</code>	0.8805	High
<code>prompt.main_instruction</code>	0.8662	High
<code>prompt.context.role</code>	0.7857	Moderate/High

Table 6.2: Segmentation performance (field-level F1) for RQ_2 .

The model demonstrates exceptional performance in isolating Examples (F1=1.0) and Data (F1=0.88). This is a critical result for analyzing Few-Shot and Data-based prompts, as it ensures that the examples and external knowledge are not confused with the main instructions. The segmentation of the Main Instruction (F1=0.86) is also reliable, indicating the tool can effectively extract the “core intent” of a prompt.

Summary for RQ_2

The automated extraction tool demonstrates high effectiveness in decomposing prompts into their macro-components. The model achieved perfect segmentation for few-shot learning scenarios ($F1 = 1.0$ for Examples) and strong performance in separating operational logic from informational content ($F1 > 0.86$ for Main Instruction and Data). This indicates that the tool is highly reliable for identifying the core functional blocks of complex prompts.

6.3 Results for RQ_3

RQ_3 investigates the model’s ability to distinguish between semantically similar but functionally distinct components, specifically focusing on the boundary between Formats and Constraints. Table 6.3 presents the performance for these subtler categories.

PST Component	F1 Score	Performance Tier
<code>prompt.context.format</code>	0.5928	Low
<code>prompt.context.constraints</code>	0.3453	Low

Table 6.3: Semantic disambiguation performance (field-level F1) for RQ_3 .

6.3.1 The “Format vs. Constraint” Challenge

As hypothesized, the model struggles significantly to disambiguate these categories, with constraints achieving the lowest score of 0.3453. While macro-segmentation (RQ_2) is robust, the fine-grained distinction between a “positive instruction for layout” (Format) and a “restrictive boundary” (Constraint) proves difficult.

6.3.2 Confusion Matrix and Qualitative Analysis

Quantitative analysis via the Confusion Matrix reveals that the primary source of error lies in the misclassification between `context.format` and `context.constraints`,

as well as between `context.data` and `context.constraints`. To better understand the nature of these errors, we conducted a qualitative inspection of the misclassified instances from the test set.

Analysis of Failure Cases

The inspection highlights two recurring patterns of ambiguity: the conflation of technical stacks with formatting styles, and the confusion between problem context and logical constraints.

Case Study 1: Format vs. Constraint

A frequent point of failure occurs when a prompt specifies a programming language or library. While the annotation guidelines classify these as Constraints (limiting the solution space), the model often perceives them as Formats (dictating the output style).

Consider the following example from the test set:

- **Prompt:** “give me a sample single page app using vue.js”
- **Ground Truth (Human Label):**
 - **Main Instruction:** "give me a sample single page app"
 - **Constraints:** "using vue.js"
- **Model Prediction:**
 - **Main Instruction:** "give me a sample single page app"
 - **Format:** "using vue.js"

Here, the model correctly isolates the text segment but mislabels the category. From a linguistic perspective, "using vue.js" functions adverbially, modifying how the app should be built. The model likely interprets this as a stylistic instruction (Format), similar to "using JSON". However, in the PST framework, a specific technology stack is a hard Constraint. This error highlights a "Construct Validity" threat: the

boundary between how something is written (Format) and what tools are used (Constraint) is semantically porous in natural language, leading to lower disambiguation performance.

Case Study 2: Data vs. Constraint

The second pattern involves complex technical queries where the user describes a bug or a scenario. The model tends to interpret the conditions of the bug as constraints on the answer.

- **Prompt:** “Why might images not show when using caching with AsyncStorage in a React Native Expo project?”
- **Ground Truth (Human Label):**
 - **Main Instruction:** "Why might images not show"
 - **Data:** "when using caching with AsyncStorage in a React Native Expo project?"
- **Model Prediction:**
 - **Main Instruction:** "Why might images not show"
 - **Constraints:** "when using caching with AsyncStorage in a React Native Expo project?"

In this instance, the Ground Truth treats the "when using..." clause as Data, it is the context or scenario that the model must analyze. The model, however, labeled it as a Constraint. This is a plausible hallucination of intent: the model interprets the scenario as a boundary condition ("Limit your answer to cases involving AsyncStorage"). This suggests that the 8B parameter model struggles to distinguish between informational context (Data) and directive boundaries (Constraints) when they are syntactically intertwined within a single sentence.

These findings align with the **Emergent Abilities** literature [33] discussed in Chapter 4. While the model is highly effective at extracting content (achieving perfect coverage), its struggle to disentangle subtle semantic nuances suggests that the 8B

parameter architecture may lack the semantic depth required to reliably distinguish between overlapping linguistic definitions, such as restriction versus formatting or scenario description versus constraint, without further scale. Consequently, future iterations of the dataset may require more rigid definition guidelines or annotation strategies to provide the explicit reinforcement necessary to bridge this semantic gap.

Summary for RQ₃

While the model demonstrates high proficiency in macro-segmentation, it exhibits significant limitations in disambiguating semantically adjacent categories. The quantitative analysis reveals a notable performance drop for **Format** ($F1 = 0.59$) and **Constraints** ($F1 = 0.34$). Qualitative inspection confirms that these errors stem from a lack of "semantic depth": the model frequently conflates stylistic directives with hard technical restrictions (e.g., treating a tech stack as a format) and misinterprets descriptive scenarios as boundary conditions. These findings corroborate the **Emergent Abilities** hypothesis, suggesting that while 8B-parameter models can master the syntactic structure of the PST, they require either larger parameter scales or more explicit data engineering to reliably resolve these subtle linguistic overlaps.

CHAPTER 7

Threats to Validity

In this chapter, we discuss the threats that could potentially affect the validity of our findings and the reliability of the proposed **Prompt Syntax Tree** (PST), dividing them into three main categories: **internal validity**, **construct validity** and **external validity**.

7.1 Construct Validity

Construct validity concerns the degree to which our operational measures and experimental design accurately reflect the theoretical concepts we intend to study.

7.1.1 Expressiveness of the PST Taxonomy

A fundamental threat lies in the design of the PST structure itself. The framework assumes that all components of a prompt can be cleanly categorized into discrete buckets (e.g., *Format*, *Constraint*, *Role*). However, natural language is inherently fluid. There is a risk that the PST schema imposes a rigid hierarchy on text that is functionally ambiguous or overlapping. For instance, a sentence like “Write a short summary” acts simultaneously as a Main Instruction (summarize) and a Constraint (short).

Forcing this into a single leaf node or splitting it arbitrarily may result in a “lossy” representation of the user’s true semantic intent, potentially oversimplifying complex linguistic nuances.

7.1.2 Subjectivity in Ground Truth Generation.

A primary threat lies in the creation of the **Ground Truth dataset**. The definition of the PST components, specifically the distinction between *Format* (positive stylistic instructions) and *Constraints* (restrictive boundaries), relies on semantic interpretation. While we established strict guidelines in the System Prompt (see Section 4.5.1), natural language is inherently ambiguous. There is a risk that the manual annotation process introduced **subjective biases**, where the same instruction could be interpreted differently depending on context (e.g., “Keep it short” could be labeled as either a format instruction or a length constraint). If the ground truth contains inconsistencies, the reported error rates for semantic disambiguation (RQ_3) may partially reflect annotation noise rather than model failure.

7.2 Internal Validity

Internal validity refers to factors within the experimental design that might influence the causal relationship between the proposed solution and the observed results.

7.2.1 Model Scale and Semantic Depth

The automated decomposition tool was built using a **Llama-3-8B-Instruct** architecture. While efficient, the literature on Emergent Abilities suggests that complex reasoning and subtle semantic disambiguation often require larger parameter scales. The difficulties observed in distinguishing between Format and Constraint (as detailed in Section 6.3.1) may be an internal limitation of the chosen 8B architecture rather than a flaw in the PST framework itself. This conflation of model capability vs. framework validity is an internal threat: we cannot definitively state if the ambiguity is inherent to the taxonomy or simply a failure of the smaller model to grasp it.

7.2.2 Synthetic Data Artifacts

To ensure the model could recognize advanced paradigms like *Tree-of-Thoughts*, we augmented the training set with synthetic data generated via the **NVIDIA NIM (NVIDIA Inference Microservices) API** (Section 4.3.2). There is a risk that the model learned to recognize the specific structural artifacts or *fingerprints* of the synthetic generator rather than generalizing to the organic complexity of human-written reasoning prompts. This could inflate the performance metrics for complex categories (RQ_1), presenting an overly optimistic view of the tool’s capabilities.

7.3 External Validity

External validity concerns the extent to which the findings can be generalized to other contexts, domains, or populations.

7.3.1 Morphological and Typological Incompatibility

A deeper linguistic threat concerns the structural validity of the PST for languages with morphosyntactic systems that differ fundamentally from English. The PST relies on the assumption that *semantic components* (e.g., Main Instruction, Data, Constraints) are **syntactically separable**, so that they appear as distinct words or phrases that can be isolated into exclusive leaf nodes. However, this assumption may fail in synthetic or polysynthetic languages (e.g., Turkish, Finnish, Inuktitut) or languages with high information density like Classical Chinese, where a single atomic unit (a word or character) may simultaneously encode the action, the object, and the constraint. For example, if a writing system uses a single logogram or a fused morpheme to represent “Write-concisely”, the PST’s “Mutual Exclusivity” rule would force a lossy split, as the single token cannot be assigned to both the Instruction node and the Constraint node without duplication. This suggests that the current leaf-node definition, which relies on positional segmentation, may be structurally invalid for languages where pragmatic functions are fused morphologically rather than distributed syntactically.

7.3.2 Empirical Validation Scope (Software Engineering)

While the Prompt Syntax Tree (PST) framework was theoretically derived from a domain-agnostic taxonomy of prompting techniques (as detailed in Chapter 3), the empirical validation of the automated extraction model was deliberately scoped to the Software Engineering domain. The training dataset was derived exclusively from DevGPT, meaning the fine-tuned model has been optimized to recognize structural patterns within technical prompts (e.g., code generation, debugging). Consequently, while the PST structure is designed to be universal, the automated tool’s ability to accurately decompose prompts in non-technical domains, such as creative writing or legal analysis, has not yet been empirically verified. There is a risk that the model may struggle to generalize the learned segmentation logic to domains with significantly different vocabulary distributions, despite the theoretical universality of the tree structure.

7.4 Summary

In this chapter, we have identified threats concerning the theoretical rigidity of the PST structure and the subjectivity of manual annotation, alongside internal limitations regarding model architecture and synthetic data usage. Additionally, we acknowledge external constraints related to the intentional scoping of the study to English software engineering contexts and potential structural incompatibilities with non-analytic languages. While these factors limit immediate generalization, they represent necessary design choices to establish a baseline for this novel representation, defining clear boundaries for this thesis and charting the course for future research.

CHAPTER 8

Conclusions

This thesis addressed the growing need for rigor in the field of Prompt Engineering. Starting from the observation that prompts are often developed through “trial and error” without a formal underlying representation, we proposed a structural framework to transition this discipline from an art to a systematic engineering practice. This chapter summarizes the work undertaken, discusses the implications of the results obtained, and outlines future research directions.

8.1 Summary

The primary objective of this research was to bridge the engineering gap between unstructured natural language prompts and the rigorous requirements of software development. The literature review highlighted a critical “reproducibility crisis” and a lack of standardized tooling for prompt management. To address this, we introduced the **Prompt Syntax Tree** (PST), a formal hierarchical representation inspired by compiler theory (AST) that decomposes prompts into addressable semantic units such as *Main Instruction*, *Context*, *Examples*, and *Reasoning*.

To operationalize this framework, we developed a complete engineering pipeline:

- **Taxonomy Formalization:** we synthesized existing prompting techniques (Zero-

Shot, Few-Shot, Chain-of-Thought) into a unified JSON-based schema capable of representing complex reasoning structures without information loss.

- **Dataset Construction:** we built a supervised dataset derived from DevGPT, consisting of real-world software engineering prompts. To overcome the scarcity of complex reasoning patterns, we implemented a synthetic data augmentation strategy using seed prompts and generative AI.
- **Model Implementation:** we fine-tuned a Llama-3-8B-Instruct model to act as an automated parser, capable of translating raw text into valid PST structures.

The empirical evaluation produced promising results. The model achieved a **Structural Validity** of 100%, generating syntactically correct JSON for every test case, and a **Coverage Score** of 99.91%, indicating a near-lossless preservation of user intent. In terms of segmentation, the model demonstrated high accuracy in isolating core components like Examples ($F1 = 1.0$) and Data ($F1 = 0.88$). However, the analysis also revealed challenges in semantic disambiguation, particularly in distinguishing between Formats and Constraints ($F1 = 0.34$), highlighting the complexity of interpreting natural language directives.

8.2 Implications and Contributions

This work offers both conceptual and practical contributions to the field of **Prompt Engineering**:

- **Formalizing “Promptware”:** by treating prompts as structured objects rather than flat strings, the PST framework lays the foundation for true **Promptware Engineering**. This representation enables the development of advanced IDE tools that can lint, validate, and refactor prompts programmatically, capabilities that were previously impossible with unstructured text.
- **Enhancing Reproducibility:** the ability to decompose a prompt into its constituent parts addresses the reproducibility crisis identified in the state of the art. Researchers and practitioners can now report prompts not just as text, but as

structured schemas, clearly separating the System Instruction from the Variable Data, ensuring that experiments are verifiable and portable across different models.

- **Automated Analysis and Optimization:** the automated extraction tool allows for the massive analysis of prompt logs. Organizations can now parse historical interaction data to understand how developers structure their requests, identifying common patterns or anti-patterns (e.g., conflicting constraints) at scale. This opens the door to automated optimization pipelines where specific components of a prompt (e.g., the Role) can be A/B tested independently of the Context.

8.3 Limitations and Future Work

While this thesis establishes a baseline for structured prompt representation, several avenues for improvement and expansion remain:

- **Cross-Domain and Multilingual Generalization:** as discussed in the Threats to Validity (Chapter 7), the current system was trained and validated exclusively on English prompts within the Software Engineering domain (DevGPT). Future work must focus on curating diverse datasets that include creative, legal, and medical prompts, as well as testing the PST structure on languages with different morphosyntactic properties (e.g., Sino-Tibetan or agglutinative languages) to ensure true universality.
- **Scaling Model Architectures:** the struggle to disambiguate subtle semantic categories (Format vs. Constraint) was linked to the limitations of the 8B parameter model. Future iterations should explore fine-tuning larger architectures (e.g., Llama-3-70B) or employing Mixture-of-Experts (MoE) models to determine if increased reasoning capacity resolves these semantic ambiguities.
- **Integration into Development Workflows:** currently, the PST is a standalone analysis tool. A natural next step is to integrate this parser directly into Integrated Development Environments (IDEs) (e.g., as a VS Code extension).

This would allow developers to visualize the structure of their prompts in real-time as they write them, receiving immediate feedback on missing contexts or contradictory instructions.

- **Standardization of the Schema:** finally, the JSON schema proposed in this work is a starting point. Future research should aim to standardize this format in collaboration with the open-source community, potentially evolving the PST into a widely adopted protocol for prompt exchange and versioning, similar to how the Language Server Protocol (LSP) standardized code intelligence.

Bibliography

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Nee-lakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020. (Cited on pages v, 1, 6, 19 e 20)
- [2] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, “Better zero-shot reasoning with role-play prompting,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 4099–4113. (Cited on pages v, 20 e 21)
- [3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022. (Cited on pages v, 21 e 22)
- [4] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022. (Cited on pages v, 22 e 23)
- [5] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances*

- in neural information processing systems*, vol. 36, pp. 11 809–11 822, 2023. (Cited on pages v e 24)
- [6] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk *et al.*, “Graph of thoughts: Solving elaborate problems with large language models,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 16, 2024, pp. 17 682–17 690. (Cited on pages v e 25)
- [7] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 534–46 594, 2023. (Cited on pages v, 27 e 28)
- [8] C. Li, J. Wang, Y. Zhang, K. Zhu, W. Hou, J. Lian, F. Luo, Q. Yang, and X. Xie, “Large language models understand and can be enhanced by emotional stimuli,” *arXiv preprint arXiv:2307.11760*, 2023. (Cited on pages v, 28 e 29)
- [9] A. Singh, A. Ehtesham, G. K. Gupta, N. K. Chatta, S. Kumar, and T. T. Khoei, “Exploring prompt engineering: A systematic review with swot analysis,” *arXiv preprint arXiv:2410.12843*, 2024. (Cited on page 5)
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186. (Cited on page 6)
- [11] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020. (Cited on page 6)
- [12] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, “A survey on evaluation of large language models,” *ACM transac-*

- tions on intelligent systems and technology, vol. 15, no. 3, pp. 1–45, 2024. (Cited on page 6)
- [13] S. Maity and M. J. Saikia, “Large language models in healthcare and medical applications: A review,” *Bioengineering*, vol. 12, no. 6, p. 631, 2025. (Cited on page 6)
- [14] J. Liu, Z. Geng, R. Cao, L. Cheng, P. Bocchini, and M. Cheng, “A large language model-empowered agent for reliable and robust structural analysis,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.02938> (Cited on page 6)
- [15] V. Tripathi, U. Allu, and B. Ahmed, “The instruction gap: Llms get lost in following instruction,” *arXiv preprint arXiv:2601.03269*, 2025. (Cited on page 7)
- [16] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, “Large language models are human-level prompt engineers,” in *The eleventh international conference on learning representations*, 2022. (Cited on page 7)
- [17] A. Della Porta, S. Lambiase, and F. Palomba, “Do prompt patterns affect code quality? a first empirical assessment of chatgpt-generated code,” in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 181–192. [Online]. Available: <https://doi.org/10.1145/3756681.3756938> (Cited on pages 7, 10, 42, 43, 46 e 56)
- [18] J. White, “A prompt pattern catalog to enhance prompt engineering with chatgpt,” *arXiv preprint arXiv:2302.11382*, 2023. (Cited on pages 8, 9, 11, 12 e 15)
- [19] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff *et al.*, “The prompt report: a systematic survey of prompt engineering techniques,” *arXiv preprint arXiv:2406.06608*, 2024. (Cited on pages 8, 9, 30, 31, 32 e 33)
- [20] Z. Chen, C. Wang, W. Sun, G. Yang, X. Liu, J. M. Zhang, and Y. Liu, “Promptware engineering: Software engineering for llm prompt development,” *arXiv preprint arXiv:2503.02400*, 2025. (Cited on pages 9, 11, 12 e 15)

-
- [21] K. Cheng, N. K. Ahmed, T. L. Willke, and Y. Sun, "Structure guided prompt: Instructing large language model in multi-step reasoning by exploring graph structure of the text," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 9407–9430. (Cited on pages 9, 12, 13, 14 e 17)
- [22] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," *arXiv preprint arXiv:2402.07927*, 2024. (Cited on pages 10, 13, 15, 18 e 31)
- [23] S. Krastev, H. Sweatman, A. Sternisko, and S. Rathje, "Epistemic fragility in large language models: Prompt framing systematically modulates misinformation correction," *arXiv preprint arXiv:2511.22746*, 2025. (Cited on page 11)
- [24] X. Han, W. Zhao, N. Ding, Z. Liu, and M. Sun, "Ptr: Prompt tuning with rules for text classification," *AI Open*, vol. 3, pp. 182–192, 2022. (Cited on page 13)
- [25] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam *et al.*, "Dspy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023. (Cited on pages 14 e 17)
- [26] Z. Xing, Y. Liu, Z. Cheng, Q. Huang, D. Zhao, D. SUN, and C. Liu, "When prompt engineering meets software engineering: Cnl-p as natural and robust "apis" for human-ai interaction," in *The Thirteenth International Conference on Learning Representations*, 2025. (Cited on page 14)
- [27] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: the threats of using llms in software engineering," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 2024, pp. 102–106. (Cited on page 14)
- [28] J. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, "Why johnny can't prompt: How non-ai experts try (and fail) to design llm prompts," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*,

- ser. CHI '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3544548.3581388> (Cited on page 15)
- [29] I. Arawjo, C. Swoopes, P. Vaithilingam, M. Wattenberg, and E. L. Glassman, "Chainforge: A visual toolkit for prompt engineering and llm hypothesis testing," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–18. (Cited on page 15)
- [30] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019. (Cited on page 18)
- [31] Z. Wang, H. Zhang, C.-L. Li, J. M. Eisenschlos, V. Perot, Z. Wang, L. Miculicich, Y. Fujii, J. Shang, C.-Y. Lee *et al.*, "Chain-of-table: Evolving tables in the reasoning chain for table understanding," *arXiv preprint arXiv:2401.04398*, 2024. (Cited on page 29)
- [32] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020. (Cited on page 29)
- [33] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler *et al.*, "Emergent abilities of large language models," *arXiv preprint arXiv:2206.07682*, 2022. (Cited on pages 49 e 67)
- [34] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022. (Cited on page 50)